Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection

by

Mashud Hasan
22299053

Nirnoy Biswas
24241185

Md. Sabbir Hossain
22299038

Al Imran
231011035

Imtiaz Zaman Sami
23101551

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
[Month Year]

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.

2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.

3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.

4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

_____	_____
Mashud Hasan	Nirnoy Biswas
22299053	24241185
_____	_____
Md. Sabbir Hossain	Al Imran
22299038	231011035
_____	
Imtiaz Zaman Sami	
23101551	

Approval

The thesis titled “Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection” submitted by

1. Mashud Hasan (22299053)
2. Nirnoy Biswas (24241185)
3. Md. Sabbir Hossain (22299038)
4. Al Imran (231011035)
5. Imtiaz Zaman Sami (23101551)

of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in Summer 2026.

Examining Committee:

Supervisor: _____

(Member) Md. Sabbir Ahmed
Senior Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor: _____

(Member) Naima Tahsin Nodi
Adjunct Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator: _____

(Member) Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
Brac University

Head of Department: _____

(Chair) Dr. Sadia Hamid Kazi
Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

Machine-learning models for financial transaction fraud detection are usually trained once and evaluated on a random split of historical data. In production this picture breaks down: fraud patterns and customer behaviour change over time (concept drift), and the true label of a transaction becomes known only weeks later, when a chargeback is filed (label delay). MLOps platforms usually decide when to retrain with statistical drift detectors on the input data, but whether such detectors reflect the performance of a fraud model when labels arrive late had not been tested.

This thesis designs and evaluates a drift-aware continuous learning MLOps framework for fraud detection. Here, drift-aware means that the framework tracks drift through the performance of the model on labels as they mature, rather than relying on label-free drift alarms. The framework monitors a LightGBM model on delayed labels, retrains it on a fixed schedule with a performance safety net, selects the training window automatically, and promotes a new model only if a promotion gate confirms that it is at least as good as the live model on unseen data. It is built with MLflow, a FastAPI scoring service and a monitoring dashboard, and is evaluated by replaying three public datasets (IEEE-CIS, Sparkov and Bank Account Fraud) as live streams under label delays of 0 to 60 days, with paired bootstrap confidence intervals, walk-forward tuning, five training seeds, an equal-budget comparison and fault injection.

Drift indicators proved unreliable in both directions: on IEEE-CIS the PR-AUC of a static model fell from 0.58 to 0.39 while the drift of its scores stayed below 0.02, whereas on Sparkov and BAF strong input drift (PSI up to 4.6) caused no loss of performance. Retraining helped on every dataset when its window suited the data, but its benefit on IEEE-CIS fell from +0.078 to +0.010 PR-AUC as the label delay grew to 60 days. Tuned fairly, drift-triggered retraining was reliably worse than scheduled retraining in seven of nine comparisons and never better; with delayed labels the schedule matched or beat it also at an equal number of retrains. Automatic window selection matched the best fixed window on two datasets and came within 0.005 PR-AUC on the third, and the promotion gate blocked every model trained on corrupted data that it could observe.

Keywords: Financial fraud detection, concept drift, label delay, continuous learning, MLOps, scheduled retraining, promotion gate, LightGBM, PR-AUC

Acknowledgement

Firstly, we thank Allah, the Most High, for everything He has bestowed upon us.

We would like to express our sincere gratitude to our supervisor, Md. Sabbir Ahmed sir, and our co-supervisor, Naima Tahsin Nodi ma'am, for their guidance and support throughout this research. Their feedback helped us to sharpen the research question and to overcome the difficulties we faced during the work.

We also thank the IEEE Computational Intelligence Society and Vesta Corporation for releasing the IEEE-CIS Fraud Detection dataset, the maintainers of the Sparkov credit-card dataset, and all the researchers whose articles and open-source tools have shaped this work.

Table of Contents

Declaration . . . . . **i**

Approval . . . . . **ii**

Abstract . . . . . **iii**

Acknowledgement . . . . . **iv**

List of Figures . . . . . **viii**

List of Tables . . . . . **x**

Nomenclature . . . . . **xi**

1 Introduction **1**

2 Literature Review **2**

3 Requirements, Impacts and Constraints **3**

4 Methodology and Design **4**

4.1 Design Process 4

4.1.1 Research Design 4

4.1.2 Simulated Deployment under Label Delay 5

4.1.3 Retraining Strategies 6

4.1.4 Drift and Performance Measures 7

4.1.5 Statistical Validation 8

4.1.6 Framework Evaluation and Fault Injection 9

4.2 Design (Model) Specification 10

4.2.1 Fraud Detection Model 10

4.2.2 Framework Architecture 12

4.2.3 Monitoring on Delayed Labels 13

4.2.4 Retraining Policy and Safety Net . . . 13
4.2.5 Training and Automatic Window Selection . . . 14
4.2.6 Promotion Gate . . . 14
4.2.7 Framework Configuration . . . 15
4.3 Data Collection . . . 15
4.4 Dataset Overview . . . 16
4.4.1 IEEE-CIS Fraud Detection . . . 16
4.4.2 Sparkov Credit Card Transactions . . . 17
4.4.3 Bank Account Fraud (BAF) . . . 17
4.4.4 Target Variable and Class Imbalance . . . 17
4.4.5 Data Cleaning and Feature Engineering . . . 18
4.4.6 Time Ordering and History-Stream Split . . . 18
4.4.7 Summary of Preprocessed Data . . . 19
4.5 Project Management Plan . . . 20
4.6 Implementation of Selected Design . . . 21
4.6.1 Back End . . . 21
4.6.2 Front End . . . 22
4.6.3 Frontend-Backend Integration . . . 23

5 Result Analysis . . . 24
5.1 Performance Evaluation . . . 24
5.1.1 Testing Method . . . 24
5.1.2 Evaluation Metrics . . . 24
5.1.3 Model Performance and Drift after Deployment . . . 24
5.1.4 Retraining under Label Delay . . . 26
5.2 Analyse Design Solutions . . . 28
5.2.1 Drift-Triggered versus Scheduled Retraining . . . 28
5.2.2 Equal-Budget Comparison . . . 29
5.2.3 Framework Replay . . . 31
5.2.4 Safeguards under Fault Injection . . . 31
5.3 Final Design Adjustments . . . 33

5.3.1 Promotion Gate on Production Features . . . 33
5.3.2 Safety-Net Reference from Live Performance . . . 33
5.3.3 Automatic Training-Window Selection . . . 33
5.3.4 Scoring Service under Load . . . 34
5.4 Summary of Results . . . 35
5.5 Statistical Analysis . . . 36
5.6 Discussions . . . 38

6 Conclusion . . . 40
6.1 Summary of Findings . . . 40
6.2 Contributions to the Field . . . 40
6.3 Recommendations for Future Work . . . 41

Bibliography . . . 42

List of Figures

4.1 Methodology overview: the experiments on three datasets, the framework and its evaluation 4

4.2 Methodology of the study: from literature review and time-ordered data preparation to the retraining experiments, the framework and its evaluation 5

4.3 Architecture of the fraud detection model: a transaction's features pass through a sequence of boosted decision trees, whose summed output becomes a fraud probability and, with the validation threshold, a decision 11

4.4 The proposed continuous-learning loop, run every week on the transaction stream . 12

4.5 Components of the proposed framework and the flow of data between them 13

4.6 Preprocessing pipeline from the raw files of each dataset to the common time-ordered table 18

4.7 Evaluation protocol: each dataset is split in time into a history and a stream that is replayed week by week; settings are tuned walk-forward in 4-week folds 19

4.8 Project management plan over the three thesis stages 20

4.9 Front end: the monitoring dashboard after a replay of IEEE-CIS (summary figures and the performance of the served model) 22

4.10 Front end: decision log and model versions on the monitoring dashboard 23

5.1 Static model after deployment, in ten consecutive time bins of the test period: PR-AUC (top) and the largest PSI among the ten most important features (bottom). Each panel has its own scale 25

5.2 Gain in mean weekly PR-AUC over the static model for scheduled retraining every 14 days, by label delay, with 95% paired bootstrap intervals 27

5.3 Equal-budget comparison on IEEE-CIS (top) and BAF (bottom): the best mean weekly PR-AUC each family reaches with at most a given number of retrains (dots: individual settings) 30

5.4 Fault injection on IEEE-CIS: change in mean weekly PR-AUC against a clean replay when two retraining jobs are faulty, with the promotion gate on and off 32

5.5 Framework replays (30-day label delay): gain in mean weekly PR-AUC over the static model for each training-window choice, with 95% bootstrap intervals over weeks 34

5.6 Load test of the scoring API before and after the change to the scoring path: throughput and 95th-percentile latency with one transaction per request, and batch throughput after the change (median latency in brackets) 35

5.7 Client-identity check on IEEE-CIS at a 30-day label delay: mean weekly PR-AUC (left) and gain over the static model (right) on all stream rows, without clients with an already known fraud, and on new clients only (95% intervals) 38

List of Tables

4.1 Retraining strategies compared in the simulation (default settings) 6

4.2 Settings searched by walk-forward tuning (190 configurations) 9

4.3 Faults injected into retraining jobs to test the safeguards 10

4.4 LightGBM settings used for every model in the study 11

4.5 Settings of the framework (configs/default.toml) 15

4.6 Sources of the datasets 15

4.7 Summary of the preprocessed data 19

4.8 Datasets used in this study (after preparation) 20

4.9 Modules of the framework (Python package fraud_mlops) 21

5.1 Performance of the static model and drift of its inputs after deployment 25

5.2 Gain in mean weekly PR-AUC over the static model by label delay (retraining every 14 days; 95% intervals; bold: interval excludes zero) 26

5.3 Walk-forward tuning: each family re-tuned every four weeks on matured labels only (mean weekly PR-AUC, total retrains, and differences with 95% intervals) 28

5.4 Equal-budget comparison: each trigger setting paired with the schedule that uses the same window and no more retrains (95% intervals; retrains: trigger / schedule) 29

5.5 Framework replays at a 30-day label delay: mean weekly PR-AUC (retrains, rejected by the gate) and the offline simulation of the same policy 31

5.6 Fault injection on IEEE-CIS (two faulty retraining jobs, 30-day label delay) 32

5.7 Scoring API load test, one transaction per request, 30 s per level: before and after the change to the scoring path (same model, identical scores) 35

5.8 Answers to the research questions 36

5.9 Key differences over five training seeds (mean, and the number of seeds in which the difference is positive). Best schedule: last 60 days on IEEE-CIS, all history on Sparkov and BAF 36

5.10 IEEE-CIS stream rows by client group at a 30-day label delay, and mean weekly PR-AUC of the static model and the two schedules (gain over static in brackets) 37

Nomenclature

The next list describes several symbols and abbreviations that will be used later within the body of the document.

ADWIN	Adaptive Windowing (drift detector)
API	Application Programming Interface
CI	Confidence Interval
DDM	Drift Detection Method
FDS	Fraud Detection System
GNN	Graph Neural Network
KS	Kolmogorov-Smirnov (test)
MLOps	Machine Learning Operations
PR-AUC	Area Under the Precision-Recall Curve
PSI	Population Stability Index
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique

Chapter 1

Introduction

Chapter 2

Literature Review

Chapter 3

Requirements, Impacts and Constraints

Chapter 4

Methodology and Design

4.1 Design Process

The study follows the design process shown in Figure 4.1: experiments that measure how a deployed fraud model behaves under drift and label delay, a framework designed around their results, and an evaluation of that framework on three datasets.

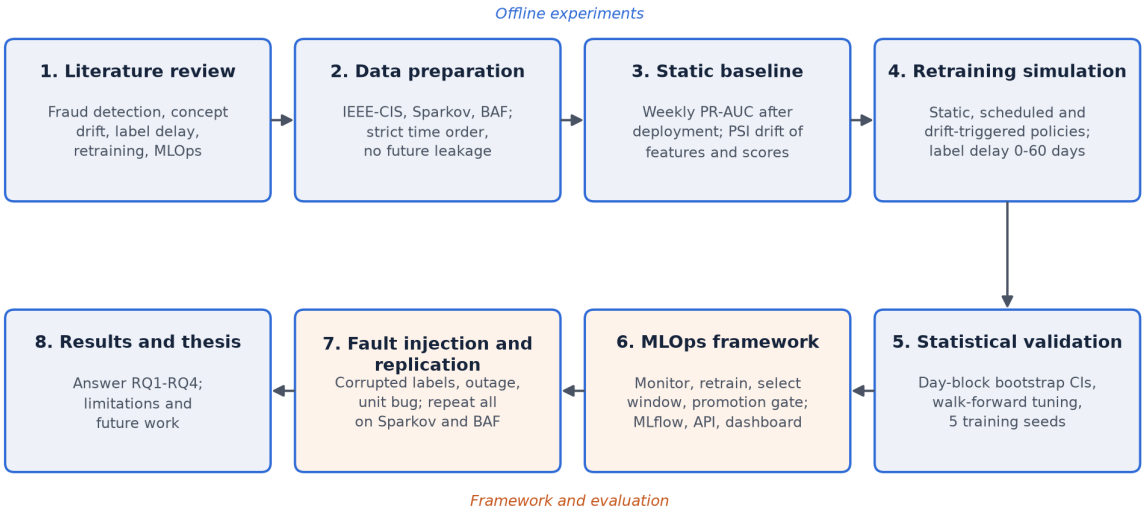


Figure 4.1: Methodology overview: the experiments on three datasets, the framework and its evaluation

4.1.1 Research Design

This study is an empirical, simulation-based investigation followed by the design and evaluation of a software framework. Chapter 2 showed that the claims made for drift-triggered retraining rest mostly on artificial drift, single datasets and labels that arrive immediately [1], [2], [3]. We therefore first measure, on public fraud data with natural drift, how a deployed model behaves and which retraining policy works best under label delay (research questions RQ1 to RQ3), and only then design the framework around the result and test it (RQ4). The study follows the workflow in Figure 4.2.

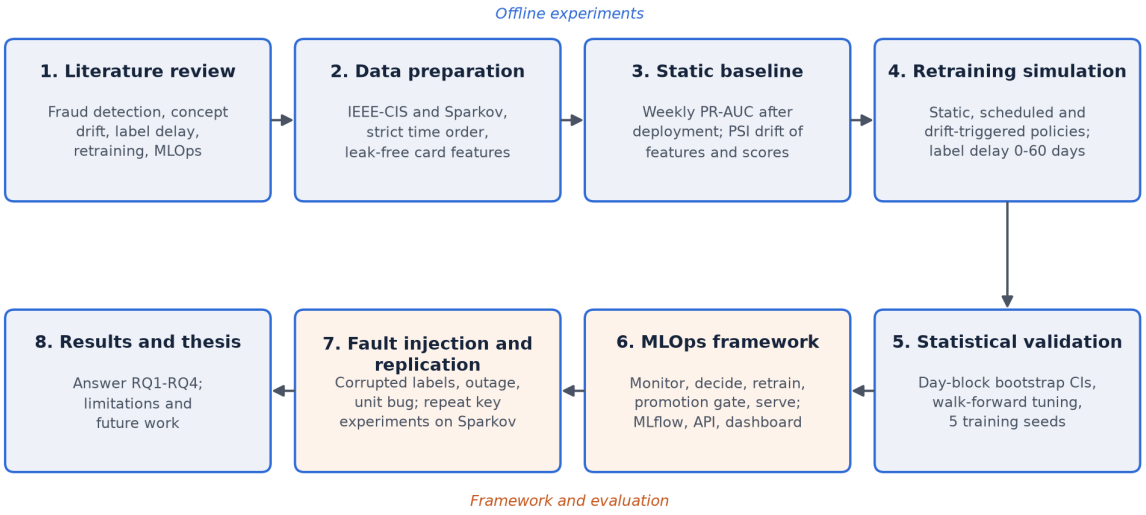


Figure 4.2: Methodology of the study: from literature review and time-ordered data preparation to the retraining experiments, the framework and its evaluation

Three principles guide every experiment. First, **time is respected**: a model is only trained on transactions that happened, and whose labels were available, before the period it is evaluated on [4]. Second, **the deployment is simulated, not imagined**: each dataset is replayed as a stream, week by week, and the label delay is applied explicitly, as described by Dal Pozzolo et al. [5]. Third, **differences must survive uncertainty**: every comparison is reported with a confidence interval, the settings of competing strategies are tuned with the same procedure, and the key results are repeated with several training seeds and on a second dataset.

The rest of this section describes the simulated deployment under label delay (Section 4.1.2), the retraining strategies (Section 4.1.3), the drift and performance measures (Section 4.1.4), the statistical validation (Section 4.1.5) and the evaluation of the framework, including fault injection (Section 4.1.6). Section 4.2 specifies the model and the framework, Sections 4.3 and 4.4 describe the data, Section 4.5 the project plan, and Section 4.6 the implementation.

4.1.2 Simulated Deployment under Label Delay

The stream is replayed in steps of seven days, from the stream start to the end of the data. At each step, three things happen in order: the strategy checks which labels are available and decides whether to retrain; if it retrains, a new model is trained and immediately replaces the old one; and the current model scores the transactions of the next seven days. The scores of each week are kept, together with the version of the model that produced them. In production the transactions would arrive through a streaming platform such as Kafka or Flink, as in the real-time pipelines of Kaushik et al. [6]; replaying recorded data instead makes every experiment exactly repeatable.

Label delay is modelled as a fixed number of days d between a transaction and the moment its label can be used. At time t, the labelled data available for monitoring and training is

$$L(t) = \{ x_j : t_j \leq t - d \} \tag{4.1}$$

where t_j is the time of transaction x_j. The first model is trained at the stream start under the same rule, so it never sees labels it could not have had. We evaluate delays of 0, 15, 30 and 60 days. A delay of 0 corresponds to the assumption made by most retraining studies [3], [2]; 30 to 60 days corresponds to the time a cardholder typically needs to notice a fraudulent charge and file a dispute [5]. Investigator feedback on a few alerted transactions, which arrives faster in a real system [5], is not modelled: every label arrives after the same delay. This makes the setting slightly pessimistic for performance-based triggers, a point we return to in the discussion.

4.1.3 Retraining Strategies

Five strategies are compared, covering the options described in Section 2.1 [7], [5]. They are summarised in Table 4.1.

Table 4.1: Retraining strategies compared in the simulation (default settings)

Strategy	When it retrains	Training data	Needs labels to decide?
Static	Never	History before the stream	-
Periodic expanding	Every 14 days	All labelled data so far	No
Periodic sliding	Every 14 days	Labelled data of the last 60 days	No
Drift: score PSI	PSI of the model's scores over the last week, against its validation scores, exceeds 0.1 (at least 14 days apart)	All labelled data so far	No
Drift: performance	PR-AUC on newly matured labels falls more than 15% below the reference (Equation 4.2; at least 14 days apart)	All labelled data so far (or the last 60 days in tuned variants)	Yes (delayed)

The **static** model is the reference: it is trained once at the stream start and never updated. The two **periodic** strategies retrain on a fixed schedule and differ only in their training data: the expanding window uses all labelled data so far, while the sliding window keeps only the most recent 60 days and forgets older patterns.

The **score-PSI** strategy is a typical label-free drift trigger [1], [2]. It compares the distribution of the scores the model produced over the last week with the scores it produced on its own validation data, using the PSI defined in Section 4.1.4, and retrains when the PSI exceeds 0.1, the usual boundary between a stable and a shifted population. It can react immediately because it needs no labels.

The **performance** strategy is an error-based trigger in the spirit of DDM and ADWIN [3], [8], adapted to delayed labels. At each step it computes the model's PR-AUC on transactions whose labels matured during the last 14 days and that the model was not trained on, provided they contain at least 30 frauds. It retrains when

$$\mathrm{AP}_{\mathrm{live}}(t) < (1 - \tau)\ \mathrm{AP}_{\mathrm{ref}} \tag{4.2}$$

where $\mathrm{AP}_{\mathrm{live}}(t)$ is that live PR-AUC, τ is the tolerance (0.15 by default), and $\mathrm{AP}_{\mathrm{ref}}$ is either the model's validation PR-AUC or its first live measurement. Both drift strategies wait at least 14 days between retrains, so that one drift episode does not trigger a series of retrains.

Each strategy is run at each label delay, with exactly the same data, model settings and random seed, so that the only difference between runs is the retraining decision and the training window. Models trained on identical windows are cached and reused across strategies, which makes the large number of runs feasible and guarantees that identical decisions give identical models.

4.1.4 Drift and Performance Measures

Population Stability Index. Drift in a feature or in the model's scores is measured with the PSI, which compares a current distribution with a reference distribution [2]:

$$\mathrm{PSI} = \Sigma_{i=1}^{B} (c_i - r_i) \ln(c_i / r_i) \tag{4.3}$$

where the reference values are divided into $B = 10$ bins at their deciles, r_i and c_i are the shares of reference and current values in bin i, missing values form a bin of their own, and shares are floored at 10^{-6} to avoid division by zero. A PSI below 0.1 is usually read as stable, 0.1 to 0.25 as a moderate shift and above 0.25 as a major shift. For the baseline analysis, the PSI is computed for the ten most important features of the static model (by total gain) and for its scores, in every time bin against the training period.

PR-AUC. Fraud is rare and investigators can only review the highest-scored alerts, so the main measure is the area under the precision-recall curve, computed as average precision [5]:

$$\mathrm{AP} = \Sigma_{n} (R_n - R_{n-1}) P_n \tag{4.4}$$

where P_n and R_n are the precision and recall at the n-th threshold of the ranked scores. Unlike accuracy, PR-AUC is not inflated by the large number of legitimate transactions, and unlike ROC-AUC it focuses on how clean the top of the alert list is [9]. A model with no skill has a PR-AUC equal to the fraud rate.

Mean weekly PR-AUC. In the stream, different weeks are scored by different model versions, whose scores are not calibrated to each other. Computing a single PR-AUC over

all weeks would mix these scales and penalise strategies that retrain often, even when every one of their models ranks transactions well. We therefore compute PR-AUC separately for each week w of the stream and report the mean over the W weeks:

$$\text{mean AP} = (1/W)\ \Sigma_{w=1}^{W}\ \text{AP}_w \tag{4.5}$$

Weeks without any fraud are skipped. The lowest weekly PR-AUC, the number of retrains and the F1-score at each model's own threshold are reported as secondary measures.

4.1.5 Statistical Validation

Paired day-block bootstrap. Weekly PR-AUC is noisy, especially when a week contains only a few hundred frauds, and consecutive transactions are correlated. Confidence intervals are therefore computed with a bootstrap that resamples whole days rather than single transactions. Within every week, the days of that week are drawn with replacement, and the mean weekly PR-AUC is recomputed. The same resampled days are used for every strategy, so differences between strategies are paired and much of the shared noise cancels out. We use 500 resamples and report 95% percentile intervals. A difference is considered reliable when its interval excludes zero.

Walk-forward tuning. A fair comparison requires that every strategy has good settings, chosen without looking at the data it is evaluated on. All 190 configurations in Table 4.2 are run over a tuning stream that starts at 35% of the rows, earlier than the evaluated stream. The evaluated stream is divided into folds of four weeks. At the start of each fold, each family (static, schedule, drift performance) selects the configuration with the highest score

$$J(c) = \text{mean AP}(c) - \kappa \times N_{\text{retrain}}(c) \,/\, (\text{number of 4-week periods}) \tag{4.6}$$

computed only on weeks whose labels have matured by the start of the fold, that is, on information that would really have been available. $\kappa = 0.002$ is a small cost per retrain that breaks ties in favour of simpler configurations. The selected configuration's results on the fold are then recorded, and the next fold repeats the selection. The result is an honest estimate of how each family would perform if an operator tuned it with the information available at the time.

Table 4.2: Settings searched by walk-forward tuning (190 configurations)

Family	Settings searched	Configurations
Static	-	1
Schedule	Expanding window, period 7, 14 or 28 days; sliding window, period 7, 14 or 28 days and window 30 or 60 days	9
Drift: performance	Tolerance 0.05, 0.1, 0.15, 0.2 or 0.3; monitoring window 7, 14 or 28 days; minimum gap 7, 14 or 28 days; reference = validation PR-AUC or first live measurement; training data = all or last 60 days	180

Training seeds. The bootstrap only covers the randomness of the evaluation sample. LightGBM also has randomness in training, through row and feature subsampling. The key configurations are therefore re-run with five random seeds (42, 1, 2, 3 and 4) at label delays of 0 and 30 days, and for each comparison we report the mean and spread of the difference and the number of seeds in which its sign holds.

Replication. Finally, the whole sequence, from the static baseline through the delay sweep, walk-forward tuning, seeds and the framework replay, is repeated unchanged on Sparkov and BAF, with the same scripts and settings. A finding that holds on IEEE-CIS but not on the other datasets is reported as dataset-specific.

4.1.6 Framework Evaluation and Fault Injection

The framework is evaluated by running it over the evaluated stream of each dataset as if it were live, with a label delay of 30 days. The replay uses the same data store, MLflow registry and model code as a deployment would. Its mean weekly PR-AUC is compared with the offline simulation of the same policy, which checks that the framework reproduces the experimental results, and with the static model. To evaluate window selection, four versions of the framework are replayed on each dataset on the same machine: a static model, a 60-day window, all history, and automatic selection. The four replays share one cache of trained models, so a model with the same training data is trained only once and the versions differ only in their decisions.

A retraining pipeline that runs automatically will eventually train on bad data. To test whether the framework's safeguards stop such models, we inject the faults in Table 4.3 into selected retraining jobs of the replay (by default the second and fourth scheduled retrains). The faults imitate common pipeline failures: a misaligned join, an outage of the chargeback feed, and a unit change between training and serving. In the first three, only the data read by the training job is corrupted, while the production feature path and the label store stay correct. The fourth corrupts the label store itself, which the gate also reads, and represents a failure the gate cannot see by design; it tests whether the safety net limits the damage.

Table 4.3: Faults injected into retraining jobs to test the safeguards

Fault	What goes wrong	Real-world cause
Label shuffle	Labels are permuted within the training window, so they no longer match their transactions	A misaligned join in the training job
Label loss	80% of frauds in the training window are recorded as legitimate	An outage of the chargeback feed
Feature unit bug	The five most important numeric features of the live model are multiplied by 100 in training only	A unit change (e.g. cents and dollars) between training and serving
Upstream label shuffle	The label store itself is corrupted, so the promotion gate also reads wrong labels	Corruption at the source; a known blind spot

Each fault scenario is replayed twice, once with the promotion gate and once without it, and compared with a clean replay. A safeguard is considered effective if the run with the fault and the safeguard stays close to the clean run, while the run without the safeguard loses performance. Each scenario uses its own MLflow store, so the main registry is never affected.

4.2 Design (Model) Specification

This section specifies the fraud detection model used throughout the study and the components of the drift-aware continuous learning framework.

4.2.1 Fraud Detection Model

All experiments use LightGBM, a gradient boosted decision tree library, with the settings in Table 4.4. Gradient boosted trees are strong and widely used models for tabular fraud data [10], [11], they handle missing values and categorical features natively, and they train fast enough to be retrained hundreds of times. Amekoe et al. also found that batch-trained boosted trees remain competitive with incremental learners when labels arrive late [11], which is exactly the setting of this study. The goal is not to find the best possible classifier, as many static studies do [12], [9], [13], but to keep one strong and realistic model fixed so that differences between retraining strategies can be attributed to the strategies alone. No resampling such as SMOTE is used, because it can distort the evaluation [14]; class imbalance is handled by weighting the fraud class instead, a cost-sensitive approach that Somasundaram and Reddy also used to handle imbalance in a drifting fraud stream [15].

Figure 4.3 shows the architecture of the model. Each transaction is described by its features, 431 on IEEE-CIS, 16 on Sparkov and 29 on BAF. LightGBM builds an ensemble of decision trees one after another, each with up to 256 leaves, and every new tree is fitted to the errors that the trees before it still make. Training stops when the validation AUC has not improved for 100 rounds, at most after 2,000 trees. For a new transaction, the outputs of all trees are summed into a score $F(x)$, which a sigmoid function turns into a fraud

probability between 0 and 1; the transaction is flagged as fraud when this probability reaches the decision threshold described below.

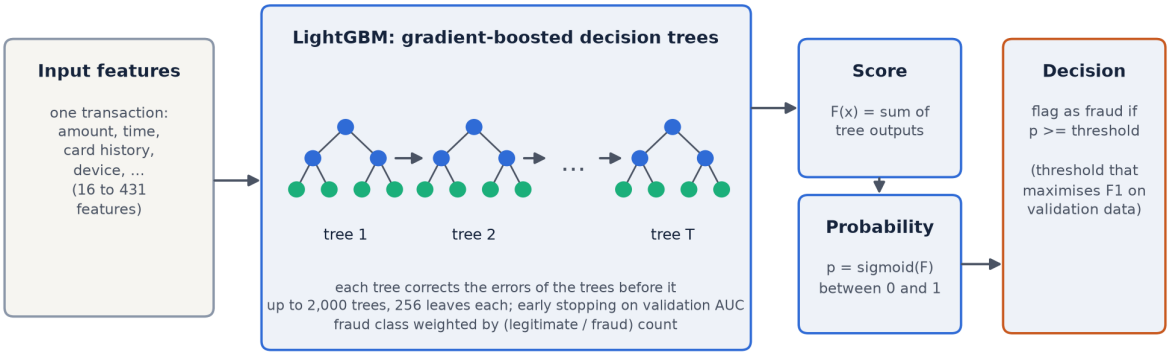


Figure 4.3: Architecture of the fraud detection model: a transaction's features pass through a sequence of boosted decision trees, whose summed output becomes a fraud probability and, with the validation threshold, a decision

Table 4.4: LightGBM settings used for every model in the study

Setting	Value	Purpose
Objective	binary	Fraud probability for each transaction
Learning rate	0.05	Step size of each boosting round
Number of leaves	256	Capacity of each tree
Minimum samples per leaf	100	Prevents leaves fitted to a few transactions
Row subsampling	0.8, every round	Randomness and regularisation
Feature subsampling	0.5 per tree	Randomness and regularisation
L2 regularisation	1.0	Shrinks leaf values
Class weight of fraud	legitimate / fraud count in the training data	Compensates for class imbalance
Boosting rounds	up to 2,000, early stopping after 100 without improvement	Stops on the validation part of the training window
Validation part	most recent 15% of the training window	Early stopping, threshold choice and reference PR-AUC

Every training run uses a time-ordered validation split: the most recent 15% of the training window is held out for early stopping, for choosing the decision threshold, and for recording the model's reference PR-AUC. The decision threshold is the one that maximises the F1-score on this validation part. It is used for the F1-score, precision and recall reported alongside PR-AUC and for the fraud flags returned by the scoring service; PR-AUC itself does not depend on a threshold. Alessi and Fugini manage the threshold dynamically

during operation to balance precision and recall [16]; in this design each model keeps the threshold chosen at training time, and a new threshold arrives with every retrained model.

For the static baseline in RQ1, the first 50% of each dataset is used for training, the next 10% for validation, and the remaining 40% is divided into ten consecutive time bins, so that performance can be followed over the months after deployment.

4.2.2 Framework Architecture

The framework turns the experimental findings into a working MLOps system. Its components are shown in Figure 4.5 and its settings in Table 4.5. It runs the loop of Figure 4.4 every seven days: monitor, decide, retrain, check, and serve. Like the closed-loop MLOps framework of Reda et al. for phishing detection [17], it connects monitoring, retraining and deployment in one automated loop; unlike it, its decisions are designed for weeks of label delay rather than near-immediate feedback, so retraining follows a schedule instead of being driven by drift events.

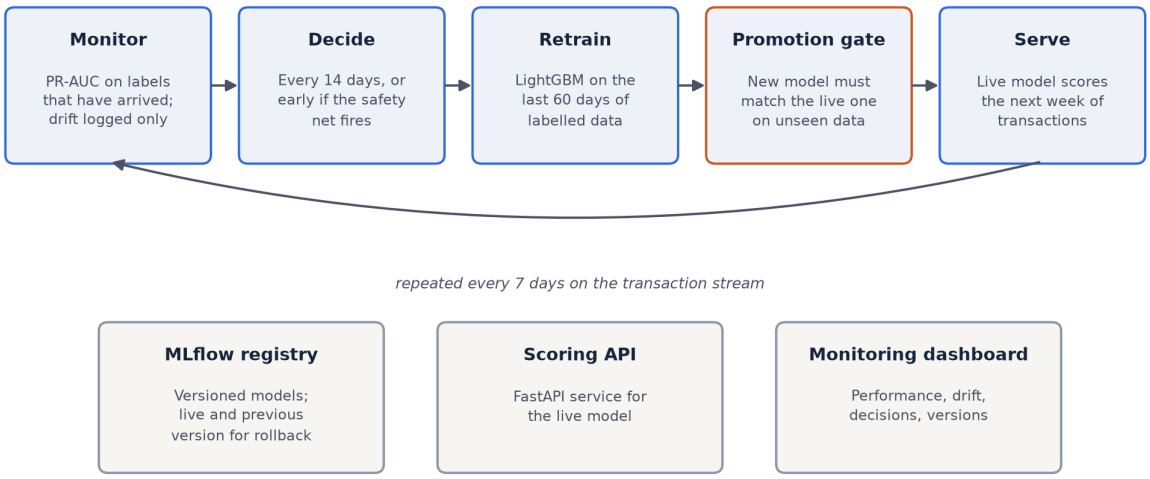


Figure 4.4: The proposed continuous-learning loop, run every week on the transaction stream

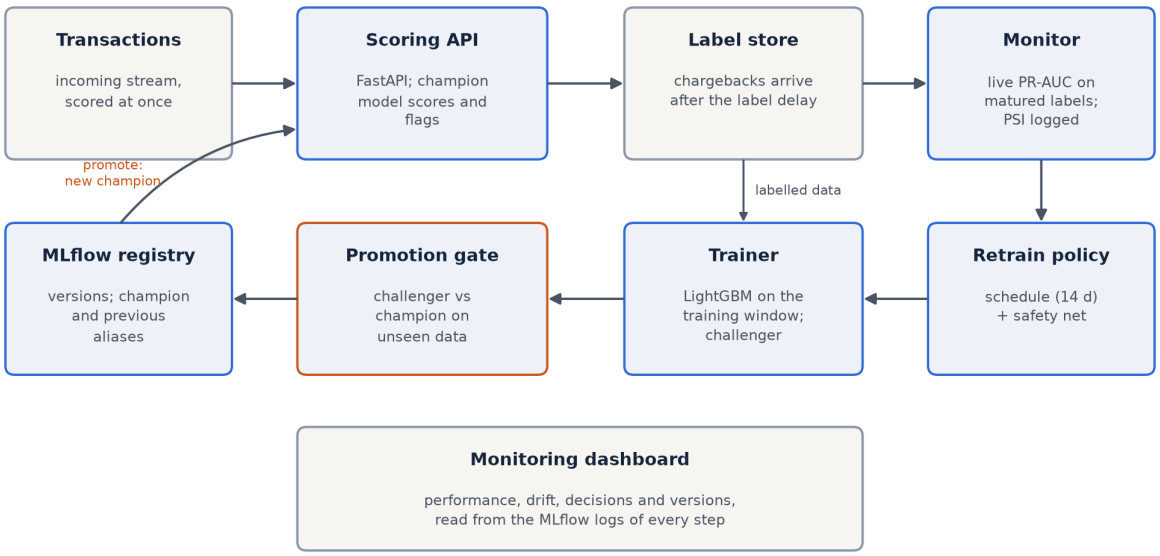


Figure 4.5: Components of the proposed framework and the flow of data between them

4.2.3 Monitoring on Delayed Labels

The monitor measures the live model's PR-AUC on the labels that matured during the last 14 days, restricted to transactions the model was not trained on. It also computes the PSI of the model's scores and of its ten most important numeric features against a sample of 50,000 rows of its training window, and logs both to MLflow. In line with the experimental results, the PSI is logged for diagnosis only and never triggers retraining on its own. Two forms of monitoring are deliberately left outside the current design: tracking drift separately for each source of data, as DriftGuard-TriAudit does for the numbers, text and layout of financial statements [18], and tracking the stability of model explanations, which John showed can become unstable and unfair as populations drift [19]. Both are natural extensions once the framework is used with several data sources or with explanations shown to investigators.

4.2.4 Retraining Policy and Safety Net

The primary policy is a fixed schedule: a new model is trained every 14 days on the last 60 days of labelled data. The schedule is complemented by a *safety net* that retrains early, at most once every seven days, when performance collapses between scheduled retrains:

$$\mathrm{AP}_{\mathrm{live}} < (1 - \tau)\,\max(\mathrm{AP}_{\mathrm{val}}, \mathrm{median}_4(\mathrm{AP}_{\mathrm{live}})) \quad \text{or} \quad \mathrm{AP}_{\mathrm{live}} < \lambda\,\pi \tag{4.7}$$

with $\tau = 0.30$, $\lambda = 3$ and π the fraud rate among the matured labels. The first condition compares live PR-AUC with the better of the model's own validation PR-AUC and the median of the last four live measurements. Using recent live history as well as the validation score matters: a model trained on corrupted data can report a poor validation score, which would otherwise lower the bar it is measured against. The second condition is

a no-skill check, since a model without skill has a PR-AUC close to the fraud rate.

4.2.5 Training and Automatic Window Selection

The trainer fits a LightGBM model with the settings of Table 4.4 on the policy's training window, which ends one label delay before the present. The window length can be set to zero for an expanding window. Each new model is registered in MLflow as a *challenger*, together with its training window, validation PR-AUC and threshold.

Automatic window selection. The experiments showed that the best training window depends on the dataset: the last 60 days on IEEE-CIS, but all labelled history on Sparkov and BAF. A fixed window chosen on one dataset can be wrong for another. The framework can therefore select the window itself. At every retrain it trains one challenger m_w per candidate window w, here the last 60 days and all labelled history, and scores all of them on the same evaluation rows E: the most recent labelled transactions that none of the candidates was trained on and the champion was not trained on either. It keeps

$$w^* = \operatorname{argmax}_{w \in W} \mathrm{AP}_E(m_w), \quad W = \{60 \text{ days, all history}\} \tag{4.8}$$

and only this challenger goes on to the promotion gate; the others are registered as not selected. The choice uses only data that would be available at the time, so it needs no knowledge of which window suits the dataset. Because the candidates are trained on identical data except for their start date, the extra cost is one additional training run per retrain.

4.2.6 Promotion Gate

A challenger replaces the live model (the *champion*) only if it passes the promotion gate, following the champion-challenger pattern of MLOps practice [7], [20]. The gate compares both models on the challenger's validation data, restricted to transactions that the champion was not trained on, so that neither model is judged on data it has seen. The challenger is promoted if

$$\mathrm{AP}(\text{challenger}) \geq \mathrm{AP}(\text{champion}) - \delta \tag{4.9}$$

with $\delta = 0.01$. Both models are re-scored through the production feature path rather than with scores stored at training time. This detail is essential: a model trained on wrongly scaled features looks good on its own, equally wrong, validation data, but fails on the inputs it will actually receive in production. If the comparison data contains fewer than 30 frauds, the comparison is not reliable and the challenger is promoted by default.

4.2.7 Framework Configuration

Table 4.5: Settings of the framework (configs/default.toml)

Setting	Value	Meaning
Loop interval	7 days	How often the framework monitors, decides and serves
Label delay d	30 days	Labels usable this long after the transaction
Retraining schedule	14 days	Primary policy
Training window	60 days	Sliding window of labelled data (0 = expanding)
Candidate windows	60 days, all	With automatic selection: one challenger per window, the best one is gated
Monitoring window	14 days	Matured labels used for live PR-AUC
Minimum frauds	30	Fewer frauds: live PR-AUC is not computed
Safety-net tolerance τ	0.30	Relative drop that triggers an early retrain
History for the reference	4	Recent live measurements in the median
No-skill factor λ	3	Live PR-AUC below 3 x fraud rate triggers
Minimum gap between retrains	7 days	Prevents repeated early retrains
Gate tolerance δ	0.01	Largest PR-AUC loss a challenger may show
Gate minimum frauds	30	Fewer frauds: the challenger is promoted by default
Drift features tracked	10	Most important numeric features (PSI logged)

4.3 Data Collection

The study uses three public fraud datasets, summarised in Table 4.6. Public data with reliable timestamps is essential for the research questions, because drift and label delay can only be studied when transactions can be replayed in the order in which they happened. Commonly used fraud datasets that cover only a few days [14], [12] were therefore not suitable.

Table 4.6: Sources of the datasets

Dataset	Source	Files used	Records in the source
IEEE-CIS	Kaggle competition IEEE-CIS Fraud Detection (IEEE Computational Intelligence Society and Vesta Corporation)	train_transaction.csv, train_identity.csv	590,540 transactions
Sparkov	Kaggle dataset Credit Card Transactions Fraud Detection (generated with the Sparkov simulator)	fraudTrain.csv, fraudTest.csv	1,852,394 transactions
BAF	Kaggle dataset Bank Account Fraud Dataset Suite (NeurIPS 2022)	Base.csv (of six variants)	1,000,000 applications

All three datasets were downloaded from Kaggle. IEEE-CIS is distributed as a competition dataset, which requires accepting the competition rules; its test files have no labels and were not used. Sparkov is distributed as two files that split one continuous period, and both were combined. BAF is distributed as a base dataset and five variants with controlled biases; the base dataset was used. The raw data is not redistributed with the code: the repository contains the scripts that prepare it, so that every result can be regenerated from the original sources.

Two practical choices made the data collection reproducible. First, every dataset is converted by a preparation script into the same table layout, so all experiments run unchanged on any of them. Second, the long experiments on the two larger replication datasets were run as Kaggle notebooks, next to the data, with the code taken directly from the public repository.

4.4 Dataset Overview

This section describes each dataset, the target variable, the cleaning and feature engineering applied to it, and the resulting preprocessed data.

4.4.1 IEEE-CIS Fraud Detection

IEEE-CIS is the main dataset. It was released by the IEEE Computational Intelligence Society and Vesta Corporation and contains 590,540 real e-commerce transactions over 182 days, of which 3.5% are fraudulent. It has been used in recent studies of drift and fraud detection [4], [10]. The transaction and identity tables are joined on the transaction identifier, giving 431 features, most of them anonymised by the provider. Only the time column (seconds from a reference point), the identifier and the label are excluded. Text columns are treated as categorical features, which LightGBM handles natively, and missing values are kept as missing rather than imputed. More than half of the columns are missing for over 50% of transactions.

4.4.2 Sparkov Credit Card Transactions

Sparkov is the first replication dataset. It contains card transactions for about 1,000 customers produced by the Sparkov transaction simulator between January 2019 and December 2020. After inspection we cut the data at 21 December 2020, because the simulator produces almost no fraud after that date (17 frauds and then none, while the volume doubles), which would make the last weeks meaningless for evaluation. The remaining 1,801,969 transactions cover 720 days with a fraud rate of 0.53%. Names, street addresses, card numbers and transaction numbers are removed. We derive the transaction hour and weekday, the customer's age, and the distance between customer and merchant, and five per-card behaviour features: the time since the card's previous transaction, its number and total amount of transactions in the previous 24 hours, its number of transactions in the previous 30 days, and the amount relative to the card's running average. Each of these is computed only from the card's *earlier* transactions, so no information from the future leaks into a feature. A lifetime transaction count was deliberately not used: it only grows over time and would drift by construction.

4.4.3 Bank Account Fraud (BAF)

BAF is the second replication dataset. It is a privacy-preserving synthetic dataset of one million bank account applications over eight months, generated from real data and designed to contain realistic drift between months. We use the Base variant. Values that the providers document as missing (negative values in six columns) are converted to missing values, a constant column is dropped, and the month itself is not used as a feature, so that the model cannot memorise the period. BAF records only the month of each application, so each record is given a random time within its month with a fixed seed. Drift therefore appears between months, while the weeks within a month are exchangeable. This is a limitation of the dataset, and the BAF results are interpreted accordingly.

4.4.4 Target Variable and Class Imbalance

The target is a binary label that marks a transaction, or an application in the case of BAF, as fraudulent (1) or legitimate (0). It is called isFraud in IEEE-CIS, is_fraud in Sparkov and fraud_bool in BAF, and is renamed isFraud in all three. Fraud is rare in every dataset: there are 27.6 legitimate transactions per fraud in IEEE-CIS, 186.1 in Sparkov and 89.7 in BAF. For this reason accuracy is not used as a measure (a model that never predicts fraud would be 96.5% to 99.5% accurate), and PR-AUC is the main measure (Section 4.1.4). The imbalance is handled by weighting the fraud class in training; no oversampling such as SMOTE is used, because resampling can make results look better than they are [14].

4.4.5 Data Cleaning and Feature Engineering

The preprocessing pipeline is shown in Figure 4.6. In IEEE-CIS, the transaction and identity tables are joined on the transaction identifier, numeric columns are stored in the smallest suitable type to reduce memory, and the 31 text columns are converted to categorical features. Missing values are kept as missing, because LightGBM handles them natively and because missingness itself can carry information: 214 of the 434 columns are missing for more than half of the transactions.

In Sparkov, names, street addresses, card numbers and transaction numbers are removed, and the data is cut at 21 December 2020, after which the simulator produces almost no fraud; this keeps 1,801,969 of 1,852,394 transactions. Time-of-day, weekday, age and the distance between customer and merchant are derived, and five per-card behaviour features are computed only from each card's earlier transactions, so that no information leaks from the future. In BAF, values documented as missing (negative values in six columns) are converted to missing, a constant column is dropped, and the month is not used as a feature.

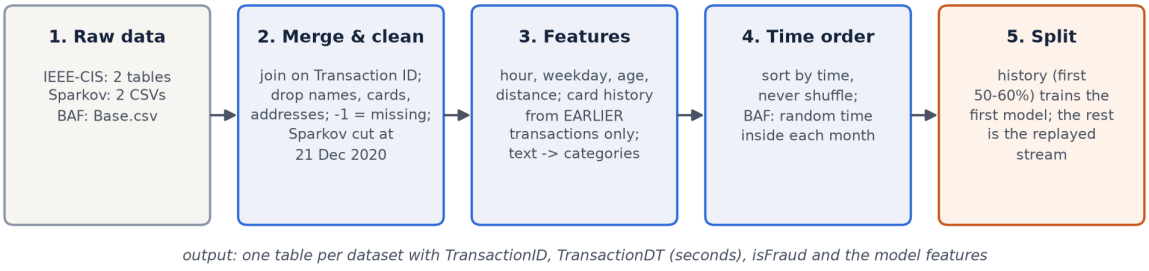


Figure 4.6: Preprocessing pipeline from the raw files of each dataset to the common time-ordered table

4.4.6 Time Ordering and History-Stream Split

All three datasets are sorted by time and never shuffled. Each is split at a fixed fraction of its rows into a *history*, used to train the first model, and a *stream*, which is replayed as if it were arriving live (Figure 4.7). The stream starts at 60% of the rows for IEEE-CIS (day 101), and at 50% for Sparkov and BAF, whose longer histories allow a full year or four months of training data before deployment.

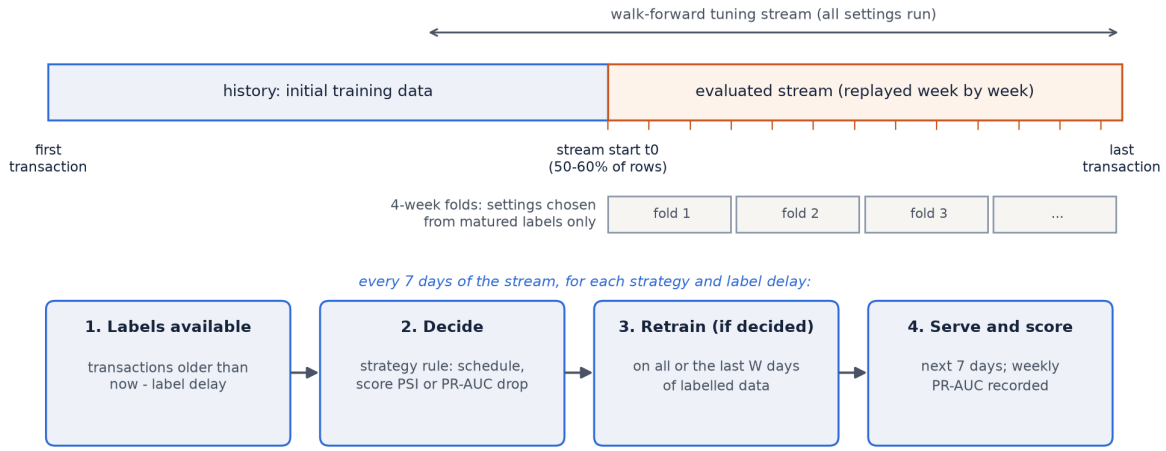


Figure 4.7: Evaluation protocol: each dataset is split in time into a history and a stream that is replayed week by week; settings are tuned walk-forward in 4-week folds

4.4.7 Summary of Preprocessed Data

Table 4.7 summarises the data after preprocessing, and Table 4.8 compares the three datasets. Every prepared dataset is a single table with a transaction identifier, a time column in seconds, the isFraud label and the model features, sorted by time.

Table 4.7: Summary of the preprocessed data

	IEEE-CIS	**Sparkov**	**BAF**
Records after preprocessing	590,540	1,801,969	1,000,000
Frauds (rate)	20,663 (3.50%)	9,634 (0.53%)	11,029 (1.10%)
Legitimate per fraud	27.6	186.1	89.7
Features (of which categorical)	431 (31)	16 (5)	29 (5)
History: training the first model	354,324 (60%)	900,984 (50%)	500,000 (50%)
Stream: replayed week by week	236,216 (12 weeks)	900,985 (52 weeks)	500,000 (19 weeks)

Table 4.8: Datasets used in this study (after preparation)

	IEEE-CIS	Sparkov	BAF (Base)
Domain	Real e-commerce card transactions (Vesta)	Simulated card transactions (Sparkov generator)	Bank account opening applications (privacy-preserving synthetic data)
Records	590,540	1,801,969	1,000,000
Time span	182 days	720 days (to 21 Dec 2020)	8 months
Fraud rate	3.50%	0.53%	1.10%
Features used	431 (transaction and identity tables, mostly anonymised)	16 (amount, time, location, customer and card-history features)	29 (applicant, request, device and velocity features)
Time resolution	Seconds from a reference point	Exact timestamp	Month only (time within the month assigned at random)
Stream starts at	60% of rows (day 101)	50% of rows (about 1 Jan 2020)	50% of rows (month 4)
Role	Main dataset	Replication	Second replication

4.5 Project Management Plan

The project was organised in three stages that match the thesis deliverables, as shown in Figure 4.8. The first stage covered the literature review, the research questions and the preparation of IEEE-CIS. The second stage covered the experiments, their statistical validation and the design of the framework. The third stage covered the framework's evaluation, the replication on two further datasets, automatic window selection and the changes requested in supervisor feedback.

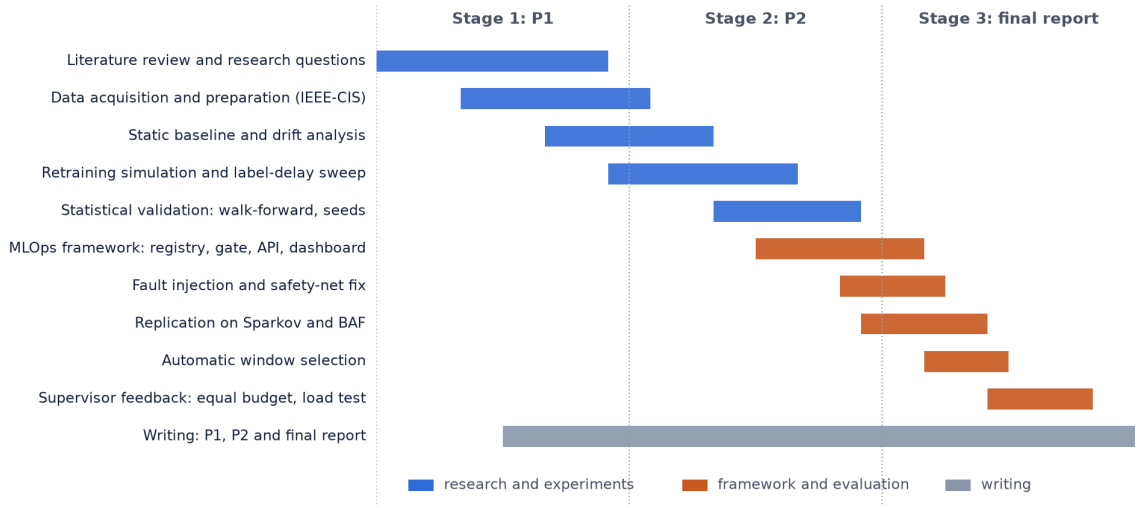


Figure 4.8: Project management plan over the three thesis stages

Work proceeded in short iterations: each experiment was planned, run, reviewed and, when its result raised a new question, followed by a further experiment. Several design decisions came out of this cycle. The walk-forward tuning was added when fixed tuning periods proved too short to rank the drift triggers; the promotion gate was changed to re-score models on production features after a fault test exposed a weakness; and automatic window selection was added when the replication showed that the best window depends on the dataset. Supervisor feedback was incorporated in the same way, for example the equal-budget comparison in Section 5.2.2.

All code and configuration were kept under version control in a Git repository hosted on GitHub, with an automated test suite run before each change. Because single experiments ran for several hours and power cuts were frequent, the long experiments save their results after every completed step and resume where they stopped; the longest runs were moved to Kaggle notebooks.

4.6 Implementation of Selected Design

The selected design was implemented as a Python package with a back end that runs the continuous learning loop, a front end for monitoring, and a model registry that connects them. Table 4.9 lists the modules.

4.6.1 Back End

Table 4.9: Modules of the framework (Python package fraud_mlops)

Module	Responsibility
config	Typed configuration loaded from one TOML file per dataset
data	Loading, caching and time-window access to the transaction table
training	LightGBM training on a window with a time-ordered validation split
model	Scoring of raw input, including missing columns and text categories
monitoring	Live PR-AUC on matured labels; PSI of scores and features
policy	Retraining policy, safety net, candidate windows and promotion gate
registry	MLflow runs and model registry with champion and previous aliases
pipeline	The replay loop: monitor, decide, retrain, gate and serve
faults	Fault injection for testing the safeguards
serving	FastAPI scoring service for the champion model
dashboard	Self-contained HTML monitoring dashboard
cli	Command line: replay, champion, dashboard and serve

The study is implemented in Python with LightGBM, pandas, NumPy and scikit-learn for the experiments, MLflow with an SQLite backend for tracking and the model registry, and FastAPI for the scoring service. The framework is organised as a Python package with

separate modules for configuration, data access, training, monitoring, the retraining policy, the promotion gate, the registry, fault injection, serving and the dashboard, and is configured with one TOML file per dataset. Twenty-four automated tests on small synthetic data check the components and the full loop.

The back end is driven from the command line. *replay* runs the loop over a dataset as if it were live, *champion* reports the model in service, *dashboard* builds the monitoring dashboard and *serve* starts the scoring service. Every training run is logged to MLflow with its parameters, metrics and training window, and every model is registered as a new version.

4.6.2 Front End

The front end is a monitoring dashboard, generated as a single HTML page from the MLflow logs of a replay, that can be opened in any browser without a server. It shows the summary figures of the run, the weekly performance of the served model together with what the loop could measure on delayed labels and the safety-net line (Figure 4.9), the drift measures, every retraining decision with its reason, and the history of model versions with the champion and the version kept for rollback (Figure 4.10).

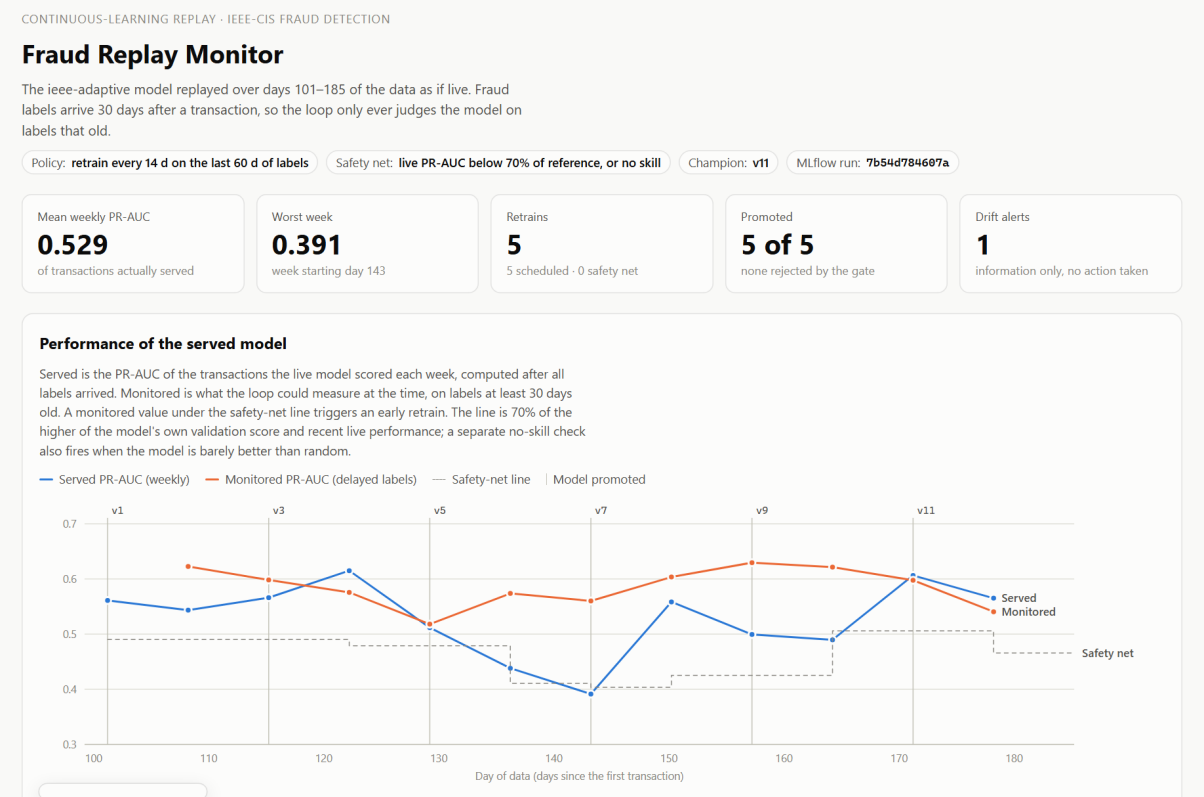


Figure 4.9: Front end: the monitoring dashboard after a replay of IEEE-CIS (summary figures and the performance of the served model)

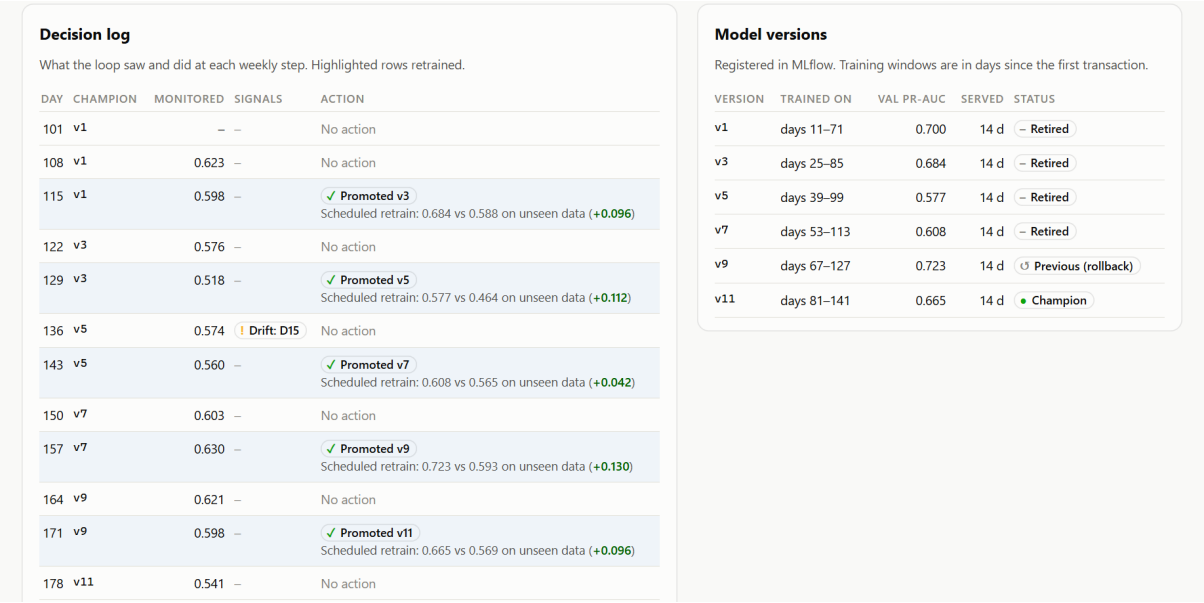


Figure 4.10: Front end: decision log and model versions on the monitoring dashboard

4.6.3 Frontend-Backend Integration

MLflow stores every model version with its parameters, metrics and training window. The model in service carries the alias *champion* and its predecessor the alias *previous*, so a rollback is a single alias change. A FastAPI service loads the champion and offers three endpoints: */health* reports the version being served, */predict* returns a fraud probability and flag for each submitted transaction, and */reload* switches to a newly promoted champion without restarting. A monitoring dashboard, generated from the MLflow logs, shows weekly performance, drift, every retraining decision with its reason, and the history of model versions.

The MLflow model registry is the contract between the parts: the back end writes model versions and moves the *champion* alias when the promotion gate accepts a challenger, the scoring service loads whatever carries that alias and reloads it on request, and the dashboard reads the same registry and run logs. No component needs to know how the others work, so the scoring service always serves the model the gate last approved.

Chapter 5

Result Analysis

5.1 Performance Evaluation

5.1.1 Testing Method

The framework and the retraining strategies are tested by replaying each dataset in strict time order, as described in Section 4.1. A model is trained on the history, deployed at the start of the stream and scores each following week before the labels of that week are known; the labels become usable only after the label delay of 0, 15, 30 or 60 days. Every strategy starts from the same initial model and sees exactly the same data, so differences between strategies come only from their retraining decisions. The framework itself is tested by a full replay with its registry, monitor and promotion gate (Section 5.2.3) and by injecting faults into its retraining jobs (Section 5.2.4).

5.1.2 Evaluation Metrics

The main metric is the mean weekly PR-AUC (Equation 4.5), computed per week so that scores of different model versions are never pooled. ROC-AUC, precision, recall and F1 at each model's validation threshold, the lowest weekly PR-AUC and the number of retrains are reported as secondary measures. Drift is measured with the PSI (Equation 4.3). Differences between strategies are reported with 95% paired day-block bootstrap intervals, and a difference is called reliable when its interval excludes zero. Because the datasets differ in difficulty, a model without skill would score 0.035 on IEEE-CIS, 0.005 on Sparkov and 0.011 on BAF, so PR-AUC values are compared within a dataset only.

5.1.3 Model Performance and Drift after Deployment

Table 5.1 and Figure 5.1 show how the static model performed over the test period of each dataset, which follows the training and validation periods in time.

Table 5.1: Performance of the static model and drift of its inputs after deployment

	IEEE-CIS	Sparkov	BAF
PR-AUC over the whole test period	0.510	0.936	0.164
ROC-AUC over the whole test period	0.888	0.997	0.871
PR-AUC, first time bin	0.581	0.954	0.164
PR-AUC, lowest time bin	0.388 (bin 6)	0.893 (bin 10)	0.149 (bin 6)
PR-AUC, last time bin	0.534	0.893	0.169
Largest feature PSI (feature)	0.17 (D10)	0.53 (card_txns_24h)	4.60 (velocity_4w)
Score-PSI trigger (threshold 0.1)	never fired; maximum score PSI 0.017	fired once (1 of 4 label delays)	never fired

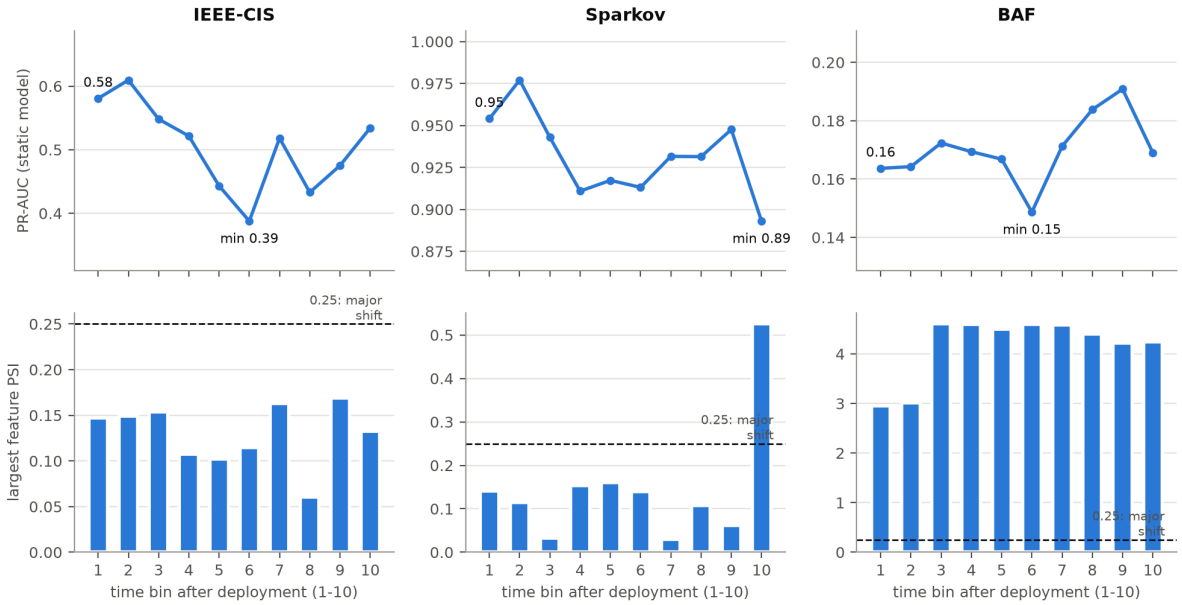


Figure 5.1: Static model after deployment, in ten consecutive time bins of the test period: PR-AUC (top) and the largest PSI among the ten most important features (bottom). Each panel has its own scale

IEEE-CIS decays, and the drift measures do not see it. The PR-AUC of the static model fell from 0.581 in the first time bin to 0.388 in the sixth, a relative loss of a third, before partly recovering to 0.534. ROC-AUC fell from 0.920 to 0.849 over the same bins. Over the same period, none of the ten most important features reached the 0.25 level of a major shift (the largest PSI was 0.17), and the PSI of the model's scores never exceeded 0.017. A label-free monitor would therefore have reported a stable system during the largest loss of performance in the study. This agrees with the decay that Menezes and Filho observed for graph models on the same dataset [4], and shows that the decay is not visible in the input distribution.

Sparkov and BAF drift, and the model does not decay. On Sparkov the per-card activity features drifted strongly in the last bin (PSI 0.53), and on BAF the velocity features shifted far beyond any usual threshold in every bin (PSI up to 4.60), yet PR-AUC stayed within 0.89 to 0.98 on Sparkov and within 0.15 to 0.19 on BAF, without a downward trend. A drift monitor on these features would have raised an alarm in almost every week. The score-PSI trigger, which looks at the model's outputs rather than its inputs, fired only once on Sparkov, at one of the four label delays, and never on BAF.

Answer to RQ1. The performance of a deployed fraud model can decay substantially, but whether it does depends on the data, and label-free drift measures are not a reliable indicator of it in either direction: they missed real decay on IEEE-CIS and signalled drift that did not matter on Sparkov and BAF. This is the situation that Shakil et al. anticipated when they noted that statistical drift does not always mean lower performance [1], now observed on natural drift in fraud data.

5.1.4 Retraining under Label Delay

Table 5.2 and Figure 5.2 compare scheduled retraining every 14 days with the static model at label delays of 0, 15, 30 and 60 days.

Table 5.2: Gain in mean weekly PR-AUC over the static model by label delay (retraining every 14 days; 95% intervals; bold: interval excludes zero)

Dataset	Delay	Static	All history	Last 60 days	Drift: performance
IEEE-CIS	0 d	0.507	**+0.033** [0.027, 0.040]	**+0.078** [0.067, 0.088]	+0.030 (2 retrains)
	15 d	0.491	**+0.025** [0.020, 0.029]	**+0.062** [0.052, 0.072]	+0.020 (2)
	30 d	0.474	**+0.028** [0.022, 0.033]	**+0.056** [0.046, 0.064]	+0.027 (3)
	60 d	0.472	**+0.008** [0.002, 0.014]	**+0.010** [0.004, 0.016]	+0.010 (4)
Sparkov	0 d	0.931	**+0.016** [0.013, 0.020]	**-0.258** [-0.273, -0.242]	0.000 (0)
	15 d	0.928	**+0.017** [0.013, 0.021]	**-0.214** [-0.228, -0.199]	0.000 (0)
	30 d	0.924	**+0.019** [0.016, 0.023]	**-0.173** [-0.187, -0.160]	0.000 (0)
	60 d	0.923	**+0.019** [0.015, 0.023]	**-0.198** [-0.211, -0.183]	0.000 (0)
BAF	0 d	0.162	**+0.011** [0.007, 0.015]	-0.003 [-0.007, 0.002]	0.000 (0)
	15 d	0.161	**+0.008** [0.004, 0.012]	-0.005 [-0.011, 0.000]	0.000 (0)
	30 d	0.161	**+0.009** [0.004, 0.014]	**-0.007** [-0.013, -0.002]	0.000 (0)
	60 d	0.155	**+0.010** [0.006, 0.015]	+0.002 [-0.003, 0.007]	+0.001 (1)

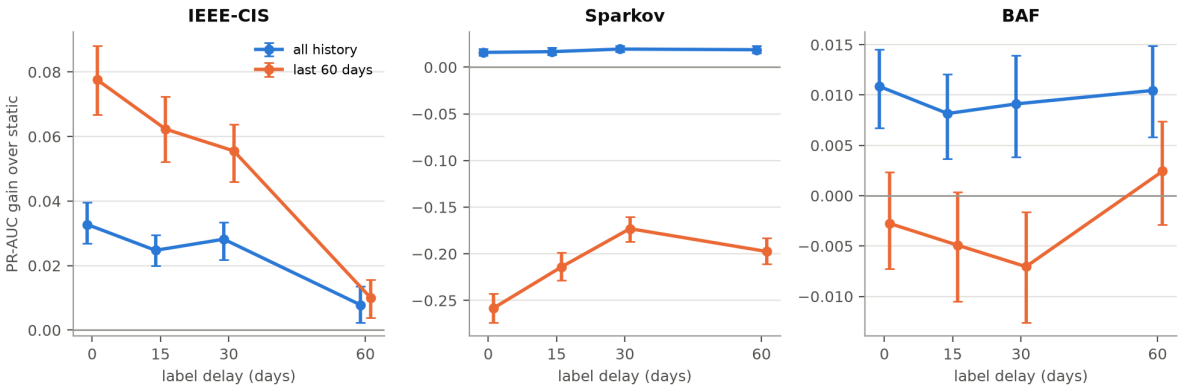


Figure 5.2: Gain in mean weekly PR-AUC over the static model for scheduled retraining every 14 days, by label delay, with 95% paired bootstrap intervals

Retraining helps when its window suits the data. On IEEE-CIS both windows improved on the static model at every delay, and the last 60 days was clearly better than all history (+0.078 against +0.033 at no delay). On Sparkov and BAF the order reversed: retraining on all history gained +0.016 to +0.019 and +0.008 to +0.011 at every delay, with every interval excluding zero, while the 60-day window was *worse* than not retraining at all, dramatically so on Sparkov (-0.17 to -0.26). Sparkov has only 0.53% fraud and a stable fraud pattern: 60 days contain too few frauds to learn it again, and the model gains nothing from forgetting older data. IEEE-CIS, in contrast, drifts, and older data teaches the model patterns that no longer hold.

Label delay erodes the benefit. On IEEE-CIS the gain of the 60-day window fell from +0.078 with immediate labels to +0.062 at 15 days, +0.056 at 30 days and +0.010 at 60 days, while the static model itself declined only from 0.507 to 0.472. With 60 days of delay, a retrained model learns from data that is already two months old, which on a drifting dataset is little better than the original training data. On Sparkov and BAF the gain of all-history retraining hardly changed with the delay, which is consistent with their lack of decay: when the pattern is stable, old labels are still useful.

Untuned drift triggers rarely act. With default settings, the performance trigger retrained two to four times on IEEE-CIS and reached between a third and half of the gain of the 60-day schedule at delays up to 30 days. On Sparkov and BAF it almost never fired, because performance never dropped by 15%, and it behaved like the static model.

Answer to RQ2. Retraining improved performance on every dataset, but only with a training window that suited the data, and on the drifting dataset its benefit shrank sharply as labels arrived later; at 60 days of delay most of the benefit was gone. This confirms, on public data, the importance of verification latency stressed by Dal Pozzolo et al. [5].

5.2 Analyse Design Solutions

5.2.1 Drift-Triggered versus Scheduled Retraining

The comparison in Section 5.1.4 used default settings. Table 5.3 gives the fair comparison: every family was re-tuned every four weeks with walk-forward tuning over 190 configurations, using only labels that had matured at the time.

Table 5.3: Walk-forward tuning: each family re-tuned every four weeks on matured labels only (mean weekly PR-AUC, total retrains, and differences with 95% intervals)

Dataset	Delay	Static	Schedule (retrains)	Schedule - static	Drift trigger - schedule
IEEE-CIS	0 d	0.470	0.614 (12)	**+0.144** [0.132, 0.158]	**-0.069** [-0.082, -0.060]
	15 d	0.468	0.547 (12)	**+0.079** [0.070, 0.091]	**-0.034** [-0.044, -0.027]
	30 d	0.458	0.518 (6)	**+0.059** [0.049, 0.069]	**-0.032** [-0.038, -0.025]
Sparkov	0 d	0.916	0.945 (12)	**+0.029** [0.024, 0.033]	-0.001 [-0.003, 0.001]
	15 d	0.913	0.944 (12)	**+0.031** [0.026, 0.036]	**-0.007** [-0.010, -0.004]
	30 d	0.917	0.943 (12)	**+0.027** [0.023, 0.031]	**-0.007** [-0.009, -0.004]
BAF	0 d	0.163	0.174 (8)	**+0.011** [0.006, 0.016]	**-0.011** [-0.016, -0.006]
	15 d	0.157	0.168 (8)	**+0.011** [0.007, 0.016]	**-0.006** [-0.011, -0.002]
	30 d	0.159	0.162 (7)	+0.003 [-0.003, 0.008]	+0.001 [-0.003, 0.006]

The tuned schedule beat or matched the tuned trigger everywhere. On IEEE-CIS the schedule was better than the drift trigger by 0.069, 0.034 and 0.032 PR-AUC at delays of 0, 15 and 30 days, all reliable. On Sparkov the trigger came close with far fewer retrains (2 to 8 against 12): the gap was within noise with immediate labels and 0.007 at 15 and 30 days. On BAF the schedule was better at 0 and 15 days, and at 30 days neither retraining family was reliably better than the static model, because the gains on BAF are small. Of the nine comparisons, the trigger was reliably worse in seven and never reliably better.

The reason is visible in how the tuned trigger behaves under label delay. It can only react to a drop in performance after the drop has been confirmed by labels that are at least one delay old, and a drop must exceed noise in weekly PR-AUC before it is acted upon. A schedule keeps the model fresh without waiting for evidence. Where the trigger came close, as on Sparkov, it did so by retraining rarely on a stable dataset where retraining matters little.

Answer to RQ3. No. When both are tuned fairly and labels arrive late, drift-triggered retraining did not outperform scheduled retraining on any of the three datasets; it was worse in most comparisons and at best equal, although it retrained less often. This contradicts the results of Wong and Perumal [2] and Yelleti [3], which were obtained with injected drift or immediate labels. A stronger effect than the choice of trigger was the choice of training window, which changed the result by up to 0.26 PR-AUC.

5.2.2 Equal-Budget Comparison

Scheduled retraining retrained more often than the drift triggers in the comparisons above (for example 12 against 2 to 5 retrains in walk-forward tuning on IEEE-CIS), so part of its advantage could come simply from retraining more often. To separate the two, both families were run over many settings on IEEE-CIS and BAF, schedules every 7 days or more and 64 trigger settings, and every trigger setting was paired with the schedule that uses the same training window and the largest number of retrains that does not exceed the trigger's. The schedule therefore never has more retrains than the trigger it is compared with, which favours the trigger. Table 5.4 summarises the pairs and Figure 5.3 shows the best PR-AUC each family reaches within a given budget.

Table 5.4: Equal-budget comparison: each trigger setting paired with the schedule that uses the same window and no more retrains (95% intervals; retrains: trigger / schedule)

Dataset	Delay	Window	Pairs	Schedule better (reliably)	Trigger reliably better	Mean diff.	Retrains
IEEE-CIS	0 d	last 60 days	27	18 (9)	0	+0.016	1.7 / 1.7
	30 d	last 60 days	30	23 (7)	0	+0.011	2.2 / 2.2
	0 d	all history	28	15 (14)	0	+0.005	1.7 / 1.6
	30 d	all history	30	12 (0)	0	0.000	2.3 / 2.2
BAF	0 d	last 60 days	9	9 (2)	0	+0.003	2.2 / 2.2
	30 d	last 60 days	0	-	-	-	trigger never fired
	0 d	all history	10	2 (1)	3	-0.002	1.9 / 1.9
	30 d	all history	2	2 (1)	0	+0.004	2.5 / 2.5

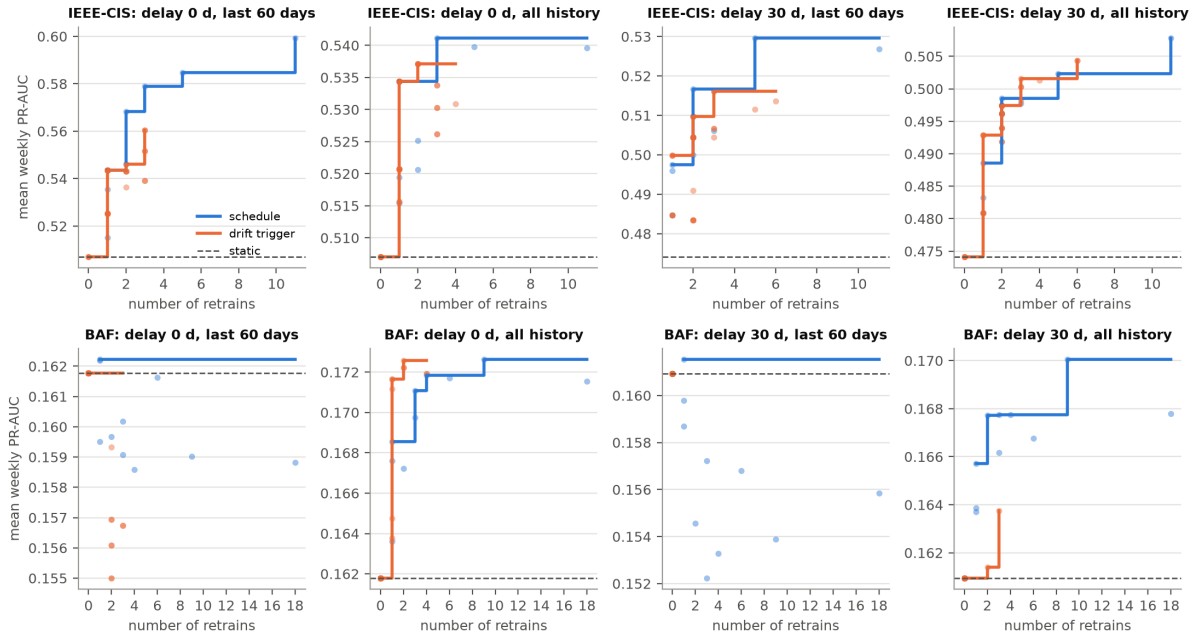


Figure 5.3: Equal-budget comparison on IEEE-CIS (top) and BAF (bottom): the best mean weekly PR-AUC each family reaches with at most a given number of retrains (dots: individual settings)

With the 60-day window, the best window on IEEE-CIS, the schedule was better in 18 of 27 pairs with immediate labels and 23 of 30 pairs at a 30-day delay, reliably so in 9 and 7 pairs, and the trigger was never reliably better; on average the schedule was 0.016 and 0.011 PR-AUC ahead at the same budget. With two retrains, for example, the best schedule reached 0.568 and the best trigger 0.546. With all history as training data the two were close: the schedule was ahead by 0.005 with immediate labels and level at a 30-day delay. The label-free score-PSI trigger fired only once across all its settings. Finally, the triggers rarely used more than two or three retrains, while the schedule kept improving as it retrained more often (0.599 with eleven retrains).

On BAF the triggers rarely fired at all, because performance hardly changed: most settings never retrained, and at a 30-day delay no setting with the 60-day window fired. Where pairs exist, the picture depends on the label delay. With immediate labels and all history as training data, the trigger was the more economical policy: it was reliably better in 3 of 10 pairs, and its best setting reached 0.173 with two retrains, which the schedule matched only with nine. With the 60-day window the schedule was better in all 9 pairs. At a realistic 30-day delay the schedule was ahead again: with two retrains it reached 0.168 against 0.161 for the best trigger, and 0.170 against 0.164 at best.

The advantage of scheduled retraining is therefore not only a matter of retraining more often. When labels were delayed by 30 days, the schedule matched or beat the drift trigger at an equal budget on both datasets, and it was never reliably worse on IEEE-CIS. A trigger can be economical when labels arrive immediately, as on BAF with all history, which is

consistent with earlier work that assumed immediate labels [3]; under label delay it has to wait for evidence of degradation and loses that advantage. Part of the schedule's overall advantage also comes from the larger number of retrains, which a delayed-label trigger cannot match. The comparison on Sparkov is being run in the same way.

5.2.3 Framework Replay

Replay. Table 5.5 reports the framework replayed over the stream of each dataset at a 30-day label delay, with four window settings run on the same Kaggle machine. The framework reproduced the offline simulation of the same policy closely: 0.529 against 0.530 for the 60-day window on IEEE-CIS, 0.942 against 0.943 for all history on Sparkov, and 0.169 against 0.170 on BAF. An earlier replay of the default configuration on a desktop computer gave 0.531 on IEEE-CIS. The MLOps components, from the registry and promotion gate to the monitoring loop, therefore add their safeguards without changing the performance of the policy they run.

Table 5.5: Framework replays at a 30-day label delay: mean weekly PR-AUC (retrains, rejected by the gate) and the offline simulation of the same policy

	IEEE-CIS (12 weeks)	**Sparkov (52 weeks)**	**BAF (19 weeks)**
Static model	0.478	0.924	0.161
Last 60 days	**0.529** (5, 0)	0.752 (26, 11)	0.153 (9, 0)
All history	0.504 (6, 0)	**0.942** (25, 0)	**0.169** (9, 1)
Automatic window	**0.529** (5, 0)	**0.942** (25, 1)	0.165 (9, 1)
Window chosen	60 days in 5 of 5	all history in 25 of 25	all history in 7 of 9
Offline simulation: last 60 days / all history	0.530 / 0.502	0.751 / 0.943	0.154 / 0.170

5.2.4 Safeguards under Fault Injection

Safeguards under fault injection. Table 5.6 and Figure 5.4 show the IEEE-CIS replay with two faulty retraining jobs. With the promotion gate on, all six faulty models produced by training-job faults were rejected. Shuffled labels gave models with a PR-AUC of 0.035, no better than chance, and the label outage gave 0.275 and 0.445 against champions of 0.475 and 0.620. The feature unit bug was the hardest case: its models scored 0.464 and 0.597 against 0.475 and 0.620, and were rejected only because they were re-scored on production features and fell just beyond the tolerance of 0.01. With the gate on, each of these three faults cost 0.012 PR-AUC, the price of keeping an older champion for two weeks. Without the gate the faulty models were promoted and the cost rose to 0.019, 0.039 and 0.080.

Table 5.6: Fault injection on IEEE-CIS (two faulty retraining jobs, 30-day label delay)

Fault	Gate	Mean PR-AUC	Change vs clean	Faulty models promoted
None (clean)	on	0.531	-	-
Label shuffle	on	0.519	-0.012	0 of 2 (0.035 vs 0.475; 0.034 vs 0.620)
	off	0.451	-0.080	2 of 2
Label loss (80%)	on	0.519	-0.012	0 of 2 (0.275 vs 0.475; 0.445 vs 0.620)
	off	0.492	-0.039	2 of 2
Feature unit bug	on	0.519	-0.012	0 of 2 (0.464 vs 0.475; 0.597 vs 0.620)
	off	0.512	-0.019	2 of 2
Upstream label shuffle	on	0.451	-0.080	2 of 2; safety net replaced each after 7 days
	off	0.451	-0.080	2 of 2

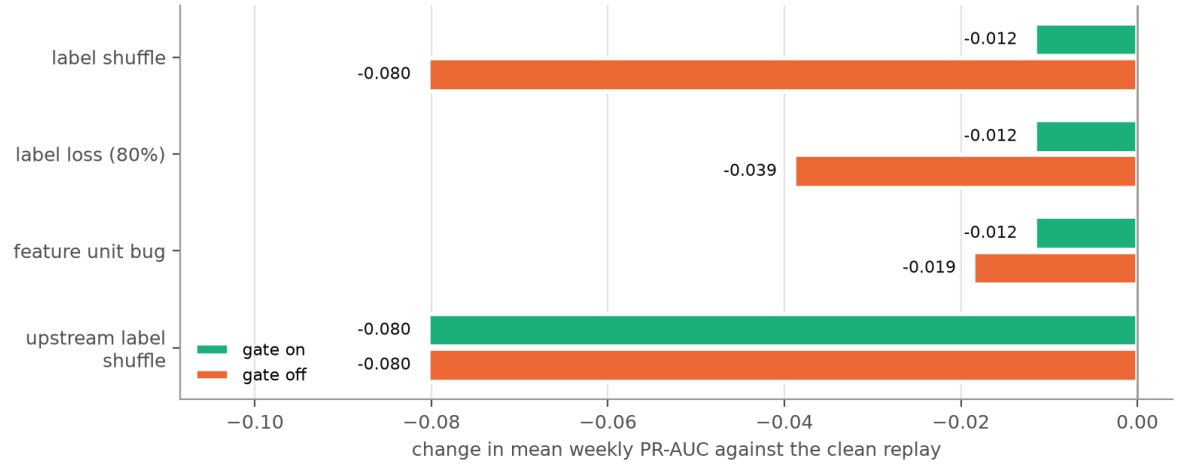


Figure 5.4: Fault injection on IEEE-CIS: change in mean weekly PR-AUC against a clean replay when two retraining jobs are faulty, with the promotion gate on and off

The upstream fault, which corrupts the label store that the gate itself reads, passed the gate as expected, since both models looked equally poor on corrupted labels. Here the safety net took over: seven days after each faulty promotion, the live PR-AUC on matured labels had fallen to 0.045 and 0.036, below both 70% of the reference and three times the fraud rate, and the framework retrained early. The damage was limited to one week per fault (-0.080 in total). Before the reference of the safety net included recent live performance (Section 4.2.4), the same fault went undetected and cost -0.149.

Answer to RQ4. Yes. Built on the findings of RQ1 to RQ3, the framework sustained the performance of the best offline policy in a full replay, prevented every faulty model it could observe from reaching production, recovered within a week from the fault it could not observe, and, with automatic window selection, performed at or near the best fixed window

on all three datasets.

5.3 Final Design Adjustments

Four changes were made to the design during the evaluation, each prompted by a result.

5.3.1 Promotion Gate on Production Features

The first version of the promotion gate compared the challenger and the champion using scores stored when each model was trained. A fault test with a feature unit bug showed the weakness of this: a model trained on wrongly scaled features looks good on equally wrong validation data. The gate was changed to re-score both models through the production feature path (Section 4.2.6), after which it rejected the faulty models.

5.3.2 Safety-Net Reference from Live Performance

Originally the safety net compared live PR-AUC with the model's own validation PR-AUC. When the label store itself was corrupted, a faulty model reported a poor validation score and was then judged against that low bar, so the fault went undetected and cost 0.149 PR-AUC. The reference was changed to the better of the validation score and the median of the last four live measurements, and a no-skill check was added (Section 4.2.4); the same fault was then caught after seven days and cost 0.080.

5.3.3 Automatic Training-Window Selection

Automatic window selection. No fixed window suited all three datasets: the 60-day window was best on IEEE-CIS (0.529) but lost 0.172 against the static model on Sparkov, while all history was best on Sparkov and BAF but only second on IEEE-CIS. With automatic selection, the framework chose the 60-day window at all five retrains on IEEE-CIS and all history at all 25 retrains on Sparkov, matching the best fixed window on both (difference 0.000 and -0.0002). On BAF it chose the 60-day window at the first two retrains, when the two candidates were close and the shorter window scored slightly higher on recent data, and all history at the remaining seven. It ended 0.0045 below the best fixed window on BAF, a small but reliable difference (95% interval over weeks -0.008 to -0.001), and 0.004 above the static model. Figure 5.5 shows that the automatic setting avoided the large loss of a wrongly chosen window on every dataset without knowing in advance which window suited the data.

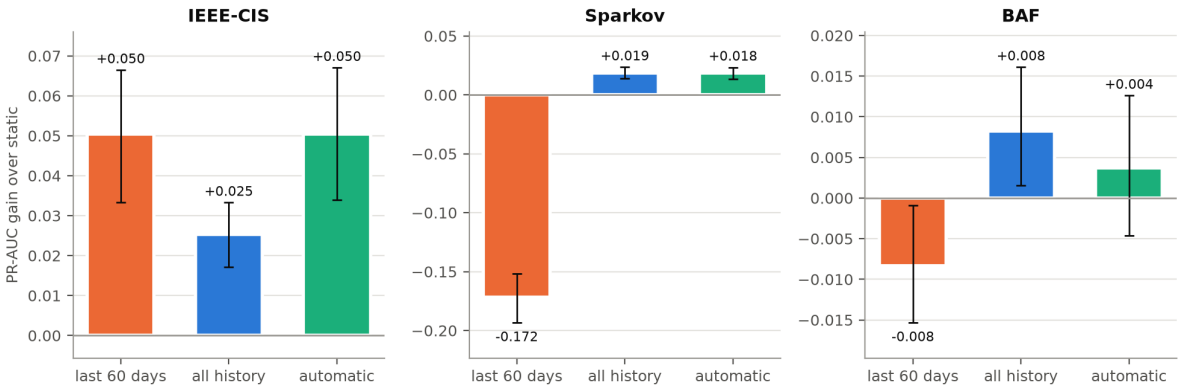


Figure 5.5: Framework replays (30-day label delay): gain in mean weekly PR-AUC over the static model for each training-window choice, with 95% bootstrap intervals over weeks

5.3.4 Scoring Service under Load

As recommended in supervisor feedback, the scoring API was load tested. The service ran as one uvicorn process with the IEEE-CIS champion model, and a client on the same machine (Intel Core i5-12400F, 6 cores and 12 threads, 16 GB memory) sent real stream transactions with all 433 raw fields to POST /predict. Each level ran for 30 seconds after a 5-second warm-up, with 1, 10, 50 and 100 concurrent clients that each wait for an answer before sending the next request, and with batches of 1 to 1,000 transactions per request. Latency was measured at the client, so it includes encoding and the HTTP round trip.

The first test showed that the service could not score in real time: it handled about 14 requests per second whatever the number of clients, and with 100 clients the 95th-percentile latency reached 8 seconds (Table 5.7). Profiling one request showed that almost all of the 57 ms went into converting the JSON input into a typed table of 433 columns and into LightGBM's per-call handling of that table, while evaluating the trees took about 0.2 ms. The scoring path was therefore changed: categorical values are mapped once to the booster's own training category codes, requests below 1,000 transactions are converted directly from the JSON records into a numeric matrix, and small requests are scored on one thread so that concurrent requests do not compete for the same cores. The scores are identical to the earlier path (maximum difference 0 on 20,000 stream transactions, with unknown categories, missing fields and text in numeric fields covered by a unit test).

Table 5.7: Scoring API load test, one transaction per request, 30 s per level: before and after the change to the scoring path (same model, identical scores)

Concurrent clients	Requests/s before	Requests/s after	p50 ms before	p50 ms after	p95 ms before	p95 ms after
1	18	810	57	1.2	61	1.7
10	14	1,314	693	7.6	938	8.9
50	14	1,323	3,478	37	4,523	41
100	14	1,295	7,075	76	7,966	82

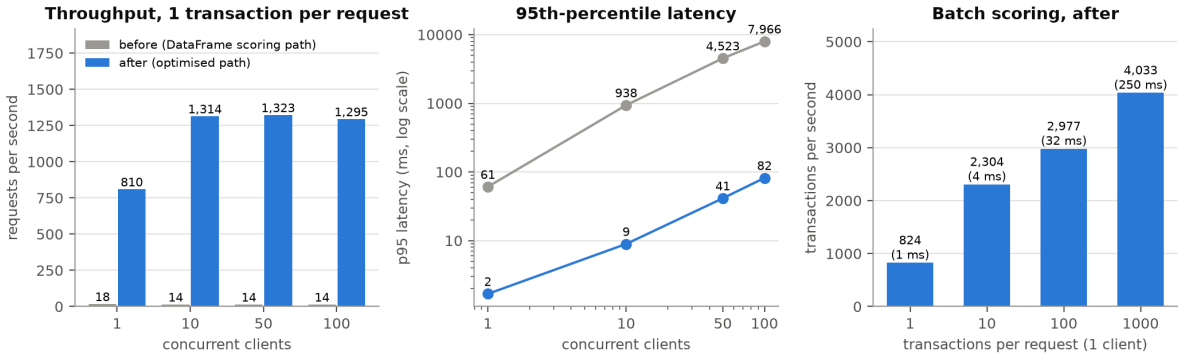


Figure 5.6: Load test of the scoring API before and after the change to the scoring path: throughput and 95th-percentile latency with one transaction per request, and batch throughput after the change (median latency in brackets)

After the change a single request took 1.2 ms at the median, the service reached about 1,300 requests per second from 10 clients on, and the 95th-percentile latency stayed below 10 ms with 10 clients and at 82 ms with 100, with no failed requests. Batching raised throughput to about 4,000 transactions per second at 1,000 transactions per request. Throughput levels off at about 1,300 requests per second because a single process handles the Python part of each request one at a time; more worker processes would scale it further. As the client shared the machine with the service, the figures are a conservative estimate for this hardware.

5.4 Summary of Results

Table 5.8 summarises the answers to the research questions. Taken together, the results support the design of the proposed framework: performance is monitored on delayed labels rather than inferred from drift, retraining follows a schedule rather than a drift alarm, the training window is selected from the data, and every new model must pass a promotion gate. Section 5.6 discusses what these results mean, how they relate to earlier work, and where they are limited.

Table 5.8: Answers to the research questions

RQ	Answer
RQ1	Performance after deployment changed differently on each dataset: it fell by a third on IEEE-CIS but stayed stable on Sparkov and BAF. Label-free drift measures did not reflect this: input and score drift stayed low while IEEE-CIS decayed, and input drift was strong (PSI up to 4.6) while Sparkov and BAF did not.
RQ2	Scheduled retraining improved performance on every dataset when its window suited the data, with gains of 0.008 to 0.078 PR-AUC. On IEEE-CIS the gain shrank from 0.078 to 0.010 as the label delay grew from 0 to 60 days.
RQ3	No. Tuned fairly with walk-forward tuning, drift-triggered retraining was worse than scheduled retraining in 7 of 9 comparisons and never better beyond noise; it retrained less often. The training window mattered more than the trigger.
RQ4	Yes. The framework reproduced the offline results within 0.002 PR-AUC, blocked every faulty model the gate could observe, limited the damage of the unobservable fault through the safety net, and, with automatic window selection, matched the best fixed window on two datasets and came within 0.005 on the third.

5.5 Statistical Analysis

Three layers of statistical analysis support the results. First, every comparison between strategies is paired: all strategies are evaluated on the same weeks and the same bootstrap resamples, which removes most of the noise they share. Second, the paired day-block bootstrap resamples whole days within each week, which respects the correlation between transactions of the same day, and yields the 95% intervals reported throughout. Third, training randomness is covered by repeating the key comparisons with five LightGBM seeds.

Table 5.9: Key differences over five training seeds (mean, and the number of seeds in which the difference is positive). Best schedule: last 60 days on IEEE-CIS, all history on Sparkov and BAF

Dataset	Delay	All history - static	Last 60 days - static	Default drift trigger - best schedule
IEEE-CIS	0 d	+0.033 (5/5)	+0.080 (5/5)	-0.055 (0/5)
	30 d	+0.022 (5/5)	+0.046 (5/5)	-0.026 (0/5)
Sparkov	0 d	+0.012 (5/5)	-0.194 (0/5)	-0.012 (0/5)
	30 d	+0.016 (5/5)	-0.171 (0/5)	-0.016 (0/5)
BAF	0 d	+0.006 (5/5)	-0.006 (0/5)	-0.006 (0/5)
	30 d	+0.009 (5/5)	-0.007 (0/5)	-0.009 (0/5)

The results are not due to training randomness. Table 5.9 repeats the key comparisons with five LightGBM seeds. Every difference in the table had the same sign in all five seeds on every dataset. The spread of a single configuration across seeds was small compared with the differences between strategies: the median standard deviation was 0.004 on IEEE-CIS, 0.010 on Sparkov and 0.003 on BAF.

Because many comparisons are reported, a single interval that excludes zero should not be over-interpreted. The conclusions of this thesis rest instead on patterns that hold across label delays, datasets, seeds and analyses: for example, the drift trigger was reliably worse than the schedule in seven of nine walk-forward comparisons and never reliably better in any of them, and the equal-budget analysis pointed the same way whenever labels were delayed.

Client identity in IEEE-CIS. The IEEE-CIS labels mark a card as fraud after a chargeback and then also mark later transactions linked to the same account, email or billing address as fraud. The same client can therefore be fraud both in a model's training data and in the weeks it scores, and part of a model's skill may come from recognising known fraudulent clients rather than fraud patterns. To measure this, a client identifier was reconstructed in the way common in the Kaggle competition (card1, addr1 and the day the account was opened, derived from D1), and every stream row was assigned, using only labels available at its time, to one of four groups: clients with an already known fraud, clients seen before without fraud, new clients, and rows without an identifier (missing addr1 or D1). The static model and the two 14-day schedules were then evaluated on each subset at a 30-day label delay (Table 5.10, Figure 5.7).

Table 5.10: IEEE-CIS stream rows by client group at a 30-day label delay, and mean weekly PR-AUC of the static model and the two schedules (gain over static in brackets)

Rows scored	Share of rows	Share of frauds	Fraud rate	Static	Schedule, last 60 days	Schedule, all history
All rows	100%	100%	3.7%	0.474	0.530 (+0.056)	0.502 (+0.028)
Without known-fraud clients	99.1%	85.9%	3.2%	0.519	0.545 (+0.027)	0.541 (+0.022)
New clients only	51.4%	43.8%	3.1%	0.304	0.343 (+0.039)	0.335 (+0.031)
Known-fraud clients	0.9%	14.1%	60.0%	-	-	-

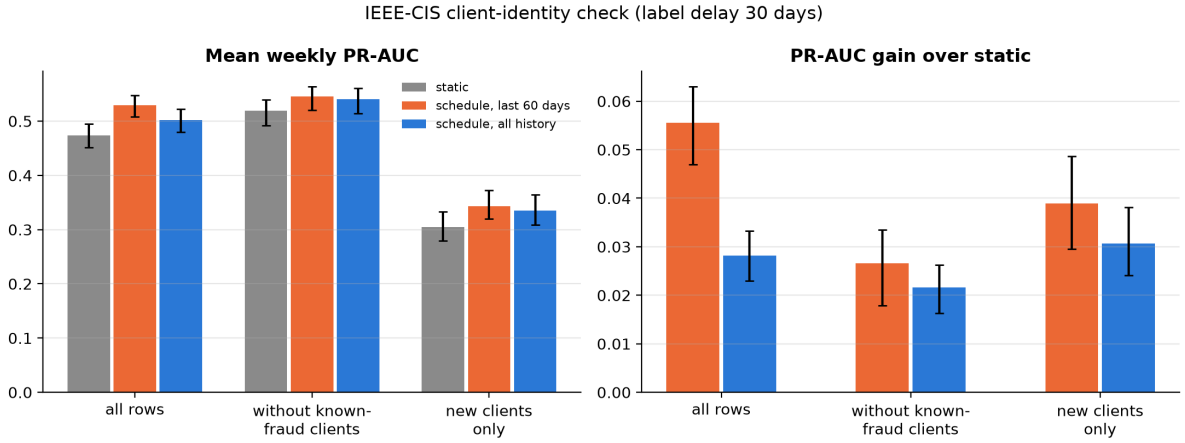


Figure 5.7: Client-identity check on IEEE-CIS at a 30-day label delay: mean weekly PR-AUC (left) and gain over the static model (right) on all stream rows, without clients with an already known fraud, and on new clients only (95% intervals)

The label propagation is clearly visible: 60% of the transactions of clients with a known fraud were fraud, against 3.1% for new clients, and these clients made up 0.9% of the stream but 14% of its frauds. Removing them roughly halved the gain of the 60-day schedule over the static model, from +0.056 to +0.027 PR-AUC, because retrained models learn the frauds of clients flagged after the first model was trained; the static model's own PR-AUC rose rather than fell. The gains nevertheless remained reliably positive in every subset, including new clients only (+0.039, interval 0.030 to 0.049), and the ranking of the strategies did not change. The retraining gains reported for IEEE-CIS therefore partly reflect this property of the dataset, which a real bank that knows its charged-back clients would also exploit, but the conclusions do not depend on it. New clients were much harder to score (PR-AUC 0.30 against 0.47 overall). The identifier is an approximation, and 41% of the stream's frauds fall in rows without one, so the check covers part of the data only.

5.6 Discussions

Drift indicators and delayed labels. The central finding is that label-free drift indicators did not reflect what mattered. They stayed quiet while the IEEE-CIS model lost a third of its PR-AUC, and they signalled strong drift on Sparkov and BAF while the models were unaffected. Shakil et al. noted that statistical drift does not always mean lower performance [1]; on natural drift in fraud data we observed this in both directions. For a fraud system this means that drift alarms should inform investigation, not trigger retraining, and that the performance on matured labels is the signal to watch, even though it arrives late. This is the sense in which the framework is drift-aware.

Schedules and triggers. Earlier work reported that drift-triggered retraining matches or beats a schedule with fewer retrains [2], [3]. Those results were obtained with injected drift or immediate labels. Under realistic label delay a performance trigger can only react after a

drop has been confirmed on labels that are weeks old, and a label-free trigger reacts to the wrong signal. A schedule needs no evidence and keeps the model fresh, and the equal-budget analysis shows that under label delay its advantage is not merely a matter of retraining more often. With immediate labels, a trigger could be the more economical policy, as on BAF, which explains why earlier studies that assumed immediate labels favoured triggers. The result is consistent with Dal Pozzolo et al., who stressed that verification latency shapes how a fraud model can be updated [5], and with Amekoe et al., who found that batch models retrained on delayed labels remain competitive [11].

The training window. The choice of training window changed results by up to 0.26 PR-AUC, far more than the choice of trigger. Recent data was best on IEEE-CIS, which decays, and all history was best on Sparkov and BAF, which are stable and have few frauds. Because no fixed window suits every deployment, selecting the window from the data, as the framework now does, is a practical way to avoid a costly wrong choice.

Safe automation. Automated retraining will eventually train on bad data. The fault tests show that a promotion gate that re-scores models on production data stops faults it can observe, and that a safety net based on live performance limits the damage of faults it cannot. Closed-loop MLOps frameworks such as HAMF [17] automate the loop; this thesis adds evidence on how such safeguards behave when labels arrive late.

Limitations. The deployment is replayed from historical data, so live latency and the behaviour of a real chargeback feed are not observed, and every label arrives after the same delay, without faster investigator feedback. One model family is used throughout. IEEE-CIS covers only six months, Sparkov is simulated, and BAF records only the month of each application, so drift within a month cannot appear. In IEEE-CIS, fraud labels are propagated to linked transactions of the same client; the check in Section 5.5 shows that this roughly halves the retraining gain once clients with a known fraud are excluded, without changing the conclusions. The equal-budget comparison is complete on IEEE-CIS and BAF and under way on Sparkov, and the scoring service was load tested on one machine, with one worker process and the load generator on the same machine.

Chapter 6

Conclusion

6.1 Summary of Findings

This thesis set out to design and evaluate a drift-aware continuous learning MLOps framework for financial transaction fraud detection that identifies when a deployed model is becoming outdated, decides when retraining is necessary, and deploys a better model safely. Replaying three public datasets under realistic label delay led to five main findings:

1. Label-free drift indicators did not reflect model performance: they missed a loss of a third of the PR-AUC on IEEE-CIS and signalled strong drift on Sparkov and BAF, where performance did not fall.

2. Retraining improved performance on every dataset when its training window suited the data, but label delay eroded its benefit: on IEEE-CIS the gain fell from +0.078 PR-AUC with immediate labels to +0.010 with a 60-day delay.

3. Tuned fairly, drift-triggered retraining was never reliably better than a fixed schedule and was reliably worse in seven of nine comparisons; with delayed labels the schedule matched or beat it also at an equal retraining budget, while with immediate labels a trigger could retrain more economically.

4. The training window mattered more than the trigger, and automatic window selection matched the best fixed window on two datasets and came within 0.005 PR-AUC on the third.

5. The framework reproduced the offline results in a full replay, its promotion gate blocked every faulty model it could observe, and its safety net limited the damage of the fault it could not observe to one week.

6.2 Contributions to the Field

1. An evaluation of retraining policies for fraud detection on public data in strict time order, with explicit label delay, paired bootstrap intervals, walk-forward tuning, repeated seeds and three datasets.

2. Evidence, on natural drift, that label-free drift indicators are an unreliable trigger for retraining fraud models, and a definition of drift-awareness based on delayed-label performance instead.

3. An equal-budget comparison showing that, under label delay, the advantage of scheduled retraining is not only due to retraining more often.

4. A drift-aware MLOps framework with delayed-label monitoring, a scheduled policy with a safety net, automatic window selection and a promotion gate that re-scores models on production features, tested by fault injection.

5. Open, reproducible code with resumable experiments and Kaggle notebooks, so that every result can be regenerated from the public datasets.

6.3 Recommendations for Future Work

1. Complete the equal-budget comparison on Sparkov, as recommended in supervisor feedback, and repeat the load test on server hardware with several worker processes and a separate load generator.

2. Model faster investigator feedback alongside delayed chargebacks, which could make performance-based triggers react sooner.

3. Repeat the study with other model families, such as neural and online learners, to test whether the findings depend on gradient boosted trees.

4. Monitor the stability of model explanations under drift [19], [10] and drift in several data sources separately [18].

5. Deploy the framework on a live transaction stream with real label feeds, for example behind a streaming platform [6].

Bibliography

[1] M. N. P. Shakil, M. S. Islam, N. Noman, and M. Papini, "Feature drift-guided adaptive ML retraining: An MLOps approach for big data analytics," in *2025 IEEE International Conference on Big Data (BigData)*, Macau, China, 2025, pp. 7719-7727. doi: 10.1109/BigData66926.2025.11401445.

[2] H. M. Wong and S. Perumal, "AI-driven model-retraining architecture to sustain operational accuracy in data-drifting environments," in *2025 IEEE Symposium on Wireless Technology & Applications (ISWTA)*, Penang, Malaysia, 2025, pp. 1-6. doi: 10.1109/ISWTA68114.2025.11329530.

[3] V. Yelleti, "ROSFD: Robust online streaming fraud detection with resilience to concept drift in data streams," arXiv preprint arXiv:2504.10229, 2025.

[4] R. S. Menezes and R. H. Filho, "Semantic and structural drift in financial knowledge graphs: A robustness analysis of GNN-based fraud detectors," in *2025 IEEE International Conference on Knowledge Graph (ICKG)*, Limassol, Cyprus, 2025, pp. 285-291. doi: 10.1109/ICKG66886.2025.00044.

[5] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784-3797, 2018. doi: 10.1109/TNNLS.2017.2736643.

[6] S. Kaushik, K. K. Dokka, D. Patel, Z. Mamadiyarov, and B. Matchanova, "Real-time analytics with intelligent data pipelines and ML detection," in *2025 International Conference on Sustainability, Innovation & Technology (ICSIT)*, Nagpur, India, 2025, pp. 1-7. doi: 10.1109/ICSIT65336.2025.11295098.

[7] M. R. Pulicharla, "Detecting and addressing model drift: Automated monitoring and real-time retraining in ML pipelines," *World Journal of Advanced Research and Reviews*, vol. 3, no. 2, pp. 147-152, 2019. doi: 10.30574/wjarr.2019.3.2.0189.

[8] K. I. Al-Daoud and I. A. Abu-AlSondos, "Robust AI for financial fraud detection in the GCC: A hybrid framework for imbalance, drift, and adversarial threats," *Journal of Theoretical and Applied Electronic Commerce Research*, vol. 20, no. 2, p. 121, 2025. doi: 10.3390/jtaer20020121.

[9] A. Abd El-Naby, E. E.-D. Hemdan, and A. El-Sayed, "An efficient fraud detection framework with credit card imbalanced data in financial services," *Multimedia Tools and Applications*, vol. 82, pp. 4139-4160, 2023. doi: 10.1007/s11042-022-13434-6.

[10] M. N. Uddin and M. M. Aziz, "Shapley value-guided adaptive ensemble learning for explainable financial fraud detection with U.S. regulatory compliance validation," arXiv preprint arXiv:2604.14231, 2026.

[11] K. M. Amekoe, M. Lebbah, G. Jaffre, H. Azzag, and Z. Chelly Dagdia, "Evaluating the efficacy of instance incremental vs. batch learning in delayed label environments: An empirical study on tabular data streaming for fraud detection," arXiv preprint

arXiv:2409.10111, 2024. doi: 10.48550/arXiv.2409.10111.

[12] I. D. Mienye and Y. Sun, "A deep learning ensemble with data resampling for credit card fraud detection," *IEEE Access*, vol. 11, pp. 30628-30638, 2023. doi: 10.1109/ACCESS.2023.3262020.

[13] I. Sohony, R. Pratap, and U. Nambiar, "Ensemble learning for credit card fraud detection," in *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (CoDS-COMAD)*, 2018, pp. 289-294. doi: 10.1145/3152494.3156815.

[14] T. Dang, T. Tran, L. Tuan, and M. Tiep, "Machine learning based on resampling approaches and deep reinforcement learning for credit card fraud detection systems," *Applied Sciences*, vol. 11, no. 21, p. 10004, 2021. doi: 10.3390/app112110004.

[15] A. Somasundaram and S. Reddy, "Parallel and incremental credit card fraud detection model to handle concept drift and data imbalance," *Neural Computing and Applications*, vol. 31, pp. 3-14, 2019. doi: 10.1007/s00521-018-3633-8.

[16] G. Alessi and M. Fugini, "Adaptive real-time financial fraud detection with explainable AI tools," *Digital Threats: Research and Practice*, vol. 7, no. 1, pp. 1-31, 2026. doi: 10.1145/3794859.

[17] A. Reda, S. A. Taie, and M. E. Shaheen, "Hybrid MLOps framework for automated lifecycle management of adaptive phishing detection models," *Scientific Reports*, vol. 15, no. 1, p. 38478, 2025. doi: 10.1038/s41598-025-23600-z.

[18] Y. Hassan, "DriftGuard-TriAudit: Concept-drift-aware continual multimodal learning for evolving financial statement fraud detection," *Journal of Computing and Electronic Information Management*, vol. 22, no. 1, 2026. doi: 10.54097/5bjxgq31.

[19] S. John, "Fair and explainable credit-scoring under concept drift: Adaptive explanation frameworks for evolving populations," arXiv preprint arXiv:2511.03807, 2025.

[20] N. Kodakandla, "Data drift detection and mitigation: A comprehensive MLOps approach for real-time systems," *International Journal of Science and Research Archive*, vol. 12, no. 1, pp. 3127-3139, 2024. doi: 10.30574/ijsra.2024.12.1.0724.